9.2.2 Gaia

The Gaia[1] methodology is intended to allow an analyst to go systematically from a statement of requirements to a design that is sufficiently detailed that it can be implemented directly. Note that we view the requirements capture phase as being independent of the paradigm used for analysis and design. In applying Gaia, the analyst moves from abstract to increasingly concrete concepts. Each successive move introduces greater implementation bias, and shrinks the space of possible systems that could be implemented to satisfy the original requirements statement. (See [Jones, 1990, pp. 216–222] for a discussion of implementation bias.)

Gaia borrows some terminology and notation from object-oriented analysis and design (specifically, FUSION [Coleman et al., 1994]). However, it is not simply a naive attempt to apply such methods to agent-oriented development. Rather, it provides an agent-specific set of concepts through which a software engineer can understand and model a complex system. In particular, Gaia encourages a developer to think of building agent-based systems as a process of *organizational design*.

ORGANIZATIONAL DESIGN

The main Gaian concepts can be divided into two categories: *abstract* and *concrete* (both of which are summarized in Table 9.1). Abstract entities are those used during analysis to conceptualize the system, but which do not necessarily have any *direct* realization within the system. Concrete entities, in contrast, are used within the design process, and will typically have direct counterparts in the run-time system.

The objective of the analysis stage is to develop an understanding of the system and its structure (without reference to any implementation detail). In the Gaia case, this understanding is captured in the system's *organization*. An organization is viewed as a collection of roles that stand in certain relationships to one another and that take part in systematic, institutionalized patterns of interactions with other roles.

The idea of a system as a society is useful when thinking about the next level in the concept hierarchy: *roles*. It may seem strange to think of a computer system as being defined by a set of roles, but the idea is quite natural when adopting an organizational view of the world. Consider a human organization such as a typical company. The company has roles such as 'president', 'vice-president', and so on. Note that in a concrete *realization* of a company, these roles will be *instantiated* with actual individuals: there will be an individual who takes on the role of president, an individual who takes on the role of vice-president, and so on. However, the instantiation is not necessarily static. Throughout the company's lifetime, many individuals may take on the role of company president, for example. Also, there is not necessarily a one-to-one mapping between roles and individuals. It is not unusual (particularly in small or informally defined organizations) for one individual to take on many roles. For example, a single individual might take on the role of 'tea maker', 'mail fetcher', and so on. Conversely, there may be many individuals that take on a single role, e.g. 'salesman'.

A role is defined by four attributes: *responsibilities, permissions, activities*, and *protocols*. *Responsibilities* determine functionality and, as such, are perhaps the key attribute associated with a role. An example responsibility associated with the role of company president might be calling the shareholders' meeting every year. Responsibilities are divided into two types:

calling the shareholders' meeting every year. Responsibilities are divided into two types: *liveness properties* and *safety properties* [Pnueli, 1986]. Liveness properties intuitively state that 'something good happens'. They describe those states of affairs that an agent must bring about, given certain environmental conditions. In contrast, safety properties are *invariants*. Intuitively, a safety property states that 'nothing bad happens' (i.e. that an acceptable state of affairs is maintained across all states of execution). An example might be 'ensure that the reactor temperature always remains in the range 0–100'.

RESPONSIBILITIES

In order to realize responsibilities, a role has a set of *permissions*. Permissions are the 'rights' associated with a role. The permissions of a role thus identify the resources that are available to that role in order to realize its responsibilities. Permissions tend to be *information resources*. For example, a role might have associated with it the ability to read a particular item of information, or to modify another piece of information. A role can also have the ability to *generate* information.

PERMISSIONS

The *activities* of a role are computations associated with the role that may be carried out by the agent without interacting with other agents. Activities are thus 'private' actions, in the sense of [Shoham, 1993].

ACTIVITIES

Finally, a role is also identified with a number of *protocols*, which define the way that it can interact with other roles. For example, a 'seller' role might have the protocols 'Dutch auction' and 'English auction' associated with it; the Contract Net protocol is associated with the roles 'manager' and 'contractor' [Smith, 1980b].

PROTOCOLS

9.2.3 Tropos

The Tropos methodology [Bresciani et al., 2004] aims to give an agent-oriented view of software throughout the software development lifecycle. It provides a conceptual framework for modelling systems, based on the following concepts:

Actor An actor in Tropos is 'an entity with strategic goals and intentionality within the system or the organizational setting' [Bresciani et al., 2004, p. 206]. Associated with actors are the notions of role and position: '[a role is] an abstract characterization of the behaviour of a social actor within some specialized context … a position is a set of roles, typically played by one agent' [Bresciani et al., 2004, p. 206].

Goal A goal in Tropos represents the actors' strategic interests. A distinction is made between hard goals (typically corresponding to system functional requirements) and soft goals (typically corresponding to non-functional requirements).

Plan A plan in Tropos is a recipe for achieving a goal.

Resource Resources in Tropos can be either physical entities or information.

Dependency A dependency is a relationship between two actors, indicating that one agent needs the other to carry out some part of a plan, deliver some resource, or similar.

Capability This represents the ability of an actor to achieve some goal, or carry out some plan.

Belief This represents knowledge that actors have about their environment.

The first phase of analysis in Tropos involves developing an *actor model* and a *dependency model*. The actor model captures the key stakeholders in the system and their strategic interests in the form of their goals; the dependency model documents the dependencies between these actors. A *goal model* is then developed, which decomposes goals into their constituent parts, and, associated with this, a *plan model* captures the structure of recipes for achieving goals.

9.2.4 Prometheus

The Prometheus methodology [Padgham and Winikoff, 2004] consists of three main stages:

System specification which focuses on identifying the goals and basic functionalities of the system, and specifies the interface between the system and its environment in terms of actions and percepts (cf. discussion in Chapter 2).

Architectural design which focuses on identifying *agent types*, the *system structure*, and the *interactions* between agents.

Detailed design which first involves refining agents into their capabilities and specifying the processes in the system, and then involves the detailed design of capabilities.

Prometheus provides a rich collection of models for each stage, and detailed guidelines for each step. It also has software tool support.

9.2.5 Agent UML

Over the past three decades, many different notations and associated methodologies have been developed within the object-oriented development community (see, for example, [Booch, 1994; Coleman et al., 1994; Rumbaugh et al., 1991]). Despite many similarities between these notations and methods, there were nevertheless many fundamental inconsistencies and differences. The unified modelling language – UML – is an attempt by three of the main figures behind object-oriented analysis and design (Grady Booch, James Rumbaugh and Ivar Jacobson) to develop a single notation for modelling object-oriented systems [Booch et al., 1998]. It is important to note that UML is *not* a methodology; it is, as its name suggests, a language for documenting models of systems; associated with UML is a methodology known as the rational unified process [Booch et al., 1998, pp. 449–456].

The fact that UML is a de facto standard for object-oriented modelling promoted its rapid take-up. When looking for agent-oriented modelling languages and tools, many researchers felt that UML was the obvious place to start [Bauer et al., 2001; Depke et al., 2001; Odell et al., 2001]. The result has been a number of attempts to adapt the UML notation for modelling agent systems. Odell and colleagues have discussed several ways in which the UML notation

might usefully be extended to enable the modelling of agent systems [Bauer et al., 2001; Odell et al., 2001]. The proposed modifications include:

- support for expressing concurrent threads of interaction (e.g. broadcast messages), thus enabling UML to model such well-known agent protocols as the Contract Net;
- a notion of 'role' that extends that provided in UML, and, in particular, allows the modelling of an agent playing many roles.

Both the Object Management Group [OMG, 2001], and FIPA are currently supporting the development of UML-based notations for modelling agent systems, and there is therefore likely to be considerable work in this area.

9.2.6 Agents in Z

Luck and d'Inverno developed an agent specification framework in the Z language [Spivey, 1992], although the types of agents considered in this framework are somewhat different from those discussed throughout most of this book [d'Inverno and Luck, 2001; Luck and d'Inverno, 1995; Luck et al., 1997]. They define a four-tiered hierarchy of the entities that can exist in an agent-based system. They start with *entities*, which are inanimate objects – they have attributes (colour, weight, position) but nothing else. They then define *objects* to be entities that have capabilities (e.g. tables are entities that are capable of supporting things). *Agents* are then defined to be objects that have goals, and are thus in some sense active; finally, *autonomous agents* are defined to be agents with motivations. The idea is that a chair could be viewed as taking on my goal of supporting me when I am using it, and can hence be viewed as an agent for me. But we would not view a chair as an *autonomous* agent, since it has no motivations (and they cannot easily be attributed to it). Starting from this basic framework, Luck and d'Inverno go on to examine the various relationships that might exist between agents of different types. In [Luck et al., 1997], they examine how an agent-based system specified in their framework might be implemented. They found that there was a natural relationship between their hierarchical agent specification framework and object-oriented systems.

> The formal definitions of agents and autonomous agents rely on inheriting the properties of lower-level components. In the Z notation, this is achieved through schema inclusion. … This is easily modelled in C++ by deriving one class from another…. Thus we move from a principled but abstract theoretical framework through a more detailed, yet still formal, model of the system, down to an object-oriented implementation, preserving the hierarchical structure at each stage.
>
> [Luck et al., 1997]

The Luck–d'Inverno formalism is attractive in the way that it captures the relationships that can exist between agents. The emphasis is placed on the notion of agents acting for one another, rather than on agents as rational systems, as we discussed above. The types of agents that the approach allows us to develop are thus inherently different from the 'rational' agents discussed above. So, for example, the approach does not help us to construct agents that can interleave proactive and reactive behaviour. This is largely a result of the chosen specification language: Z. This language is inherently geared towards the specification of operation-based, functional systems. The basic language has no mechanisms that allow us to easily specify the ongoing behaviour of an agent-based system. There are, of course, extensions to Z designed

for this purpose.

Discussion

The predominant approach to developing methodologies for multiagent systems is to adapt those developed for object-oriented analysis and design [Booch, 1994]. There are several disadvantages with such approaches. First, the kinds of *decomposition* that object-oriented methods encourage is at odds with the kind of decomposition that *agent-oriented* design encourages. I discussed the relationship between agents and objects in Chapter 2; it]should be clear from this discussion that agents and objects are very different beasts. While agent systems implemented using object-oriented programming languages will typically contain many objects, they will contain far fewer agents. A good agent-oriented design methodology would encourage developers to achieve the correct decomposition of entities into either agents or objects.

AGENT-ORIENTED DECOMPOSITION

Another problem is that object-oriented methodologies simply do not allow us to capture many aspects of agent systems; for example, it is hard to capture in object models such notions as an agent proactively generating actions or dynamically reacting to changes in their environment, still less how to effectively cooperate and negotiate with other self-interested agents. The extensions to UML proposed in [Odell et al., 2001], [Depke et al., 2001] and [Bauer et al., 2001] address some, but by no means all of these deficiencies. At the heart of the problem is the problem of the relationship between agents and objects, which has not yet been satisfactorily resolved.

9.3 Pitfalls of Agent Development

In this section (summarized from [Wooldridge and Jennings, 1998]), I give an overview of some of the main pitfalls awaiting the unwary multiagent system developer.

You oversell agent solutions, or fail to understand where agents may usefully be applied Agent technology is currently the subject of considerable attention in the computer science and AI communities, and many predictions have been made about its long-term potential. However, one of the greatest current sources of perceived failure in agent-development initiatives is simply the fact that developers overestimate the potential of agent systems. While agent technology represents a potentially novel and important new way of conceptualizing and implementing software, it is important to understand its limitations. Agents are ultimately just software, and agent solutions are subject to the same fundamental limitations as more conventional software solutions. In particular, agent technology has not somehow solved the (very many) problems that have dogged AI since its inception. Agent systems typically make use of AI techniques, and, in this sense, they are an application of AI technology, but their 'intelligent' capabilities are limited by the state of the art in this field. Artificial intelligence as a field has suffered from overoptimistic claims about its potential. It seems essential that agent technology does not fall prey to this same problem: realistic expectations of what agent technology can provide are thus important.

You get religious or dogmatic about agents Although agents have been used in a wide range of applications, they are not a universal solution. There are many applications for

which conventional software development paradigms (such as object-oriented programming) are more appropriate. Indeed, given the relative immaturity of agent technology and the small number of deployed agent applications, there need to be clear advantages to an agent-based solution before such an approach is even contemplated.

You do not know why you want agents This is a common problem for any new technology that has been hyped as much as agents. Managers read optimistic financial forecasts of the potential for agent technology and, not surprisingly, they want part of this revenue. However, in many cases managers who propose an agent project do not actually have a clear idea about what 'having agents' will buy them. In short, they have no business model for agents: they have no understanding of how agents can be used to enhance their existing products; how they can enable them to generate new product lines; and so on.

You want to build generic solutions to one-off problems This is a pitfall to which many software projects fall victim, but it seems especially prevalent in the agent community. It typically manifests itself in the devising of an architecture or testbed that supposedly enables a whole range of potential types of system to be built, when what is really required is a bespoke design to tackle a single problem. In such situations, a custom-built solution will be easier to develop and far more likely to satisfy the requirements of the application.

You believe that agents are a silver bullet The holy grail of software engineering is a 'silver bullet': a technique that will provide an order of magnitude improvement in software development. Agent technology is a newly emerged, and as yet essentially untested, software paradigm, but it is only a matter of time before someone claims agents are a silver bullet. This would be dangerously naive. As we pointed out above, there are good arguments in favour of the view that agent technology will lead to improvements in the development of complex distributed software systems. But, as yet, these arguments are largely untested in practice.

You forget that you are developing software At the time of writing, the development of any agent system – however trivial – is essentially a process of experimentation. Although I discussed a number of methodologies above, there are no tried and trusted methodologies available. Unfortunately, because the process is experimental, it encourages developers to forget that they are actually developing software. The result is a foregone conclusion: the project flounders, not because of agent-specific problems, but because basic software engineering good practice was ignored.

You forget that you are developing multithreaded software Multithreaded systems have long been recognized as one of the most complex classes of computer system to design and implement. By their very nature, multiagent systems tend to be multithreaded (both within an agent and certainly within the society of agents). So, in building a multiagent system, it is vital not to ignore the lessons learned from the concurrent and distributed systems community – the problems inherent in multithreaded systems do not go away, just because you adopt an agent-based approach.

Your design does not exploit concurrency One of the most obvious features of a poor

multiagent design is that the amount of concurrent problem solving is comparatively small or even in extreme cases non-existent. If there is only ever a need for a single thread of control in a system, then the appropriateness of an agent-based solution must seriously be questioned.

You decide that you want your own agent architecture Agent architectures are essentially templates for building agents. When first attempting an agent project, there is a great temptation to imagine that no existing agent architecture meets the requirements of your problem, and that it is therefore necessary to design one from first principles. But designing an agent architecture from scratch in this way is often a mistake; my recommendation is therefore to study the various architectures described in the literature, and either license one or else implement an 'off-the-shelf' design.

Your agents use too much AI When one builds an agent application, there is an understandable temptation to focus exclusively on the agent-specific 'intelligence' aspects of the application. The result is often an agent framework that is too overburdened with experimental techniques (natural language interfaces, planners, theorem provers, reason maintenance systems, etc.) to be usable.

You see agents everywhere When one learns about multiagent systems for the first time, there is a tendency to view everything as an agent. This is perceived to be in some way conceptually pure. But if one adopts this viewpoint, then one ends up with agents for everything, including agents for addition and subtraction. It is not difficult to see that naively viewing everything as an agent in this way will be extremely inefficient: the overheads of managing agents and interagent communication will rapidly outweigh the benefits of an agent-based solution. Moreover, we do not believe it is useful to refer to very fine-grained computational entities as agents.

You have too few agents While some designers imagine a separate agent for every possible task, others appear not to recognize the value of a multiagent approach at all. They create a system that completely fails to exploit the power offered by the agent paradigm, and develop a solution with a very small number of agents doing all the work. Such solutions tend to fail the standard software engineering test of cohesion, which requires that a software module should have a single, coherent function. The result is rather as if one were to write an object-oriented program by bundling all the functionality into a single class. It can be done, but the result is not pretty.

You spend all your time implementing infrastructure One of the greatest obstacles to the wider use of agent technology is that there are no widely used software platforms for developing multiagent systems. Such platforms would provide all the basic infrastructure (for message handling, tracing and monitoring, run-time management, and so on) required to create a multiagent system. As a result, almost every multiagent system project that we have come across has had a significant portion of available resources devoted to implementing this infrastructure from scratch. During this implementation stage, valuable time (and hence money) is often spent implementing libraries and software tools that, in the end, do little more than exchange messages across a network. By the time these libraries and tools have been implemented, there is frequently little time, energy, or enthusiasm left to work either on the agents themselves, or on the cooperative/social

aspects of the system.

Your agents interact too freely or in an disorganized way The dynamics of multiagent systems are complex, and can be chaotic. Often, the only way to find out what is likely to happen is to run the system repeatedly. If a system contains many agents, then the dynamics can become too complex to manage effectively. Another common misconception is that agent-based systems require no real structure. While this may be true in certain cases, most agent systems require considerably more system-level engineering than this. Some way of structuring the society is typically needed to reduce the system's complexity, to increase the system's efficiency, and to more accurately model the problem being tackled.

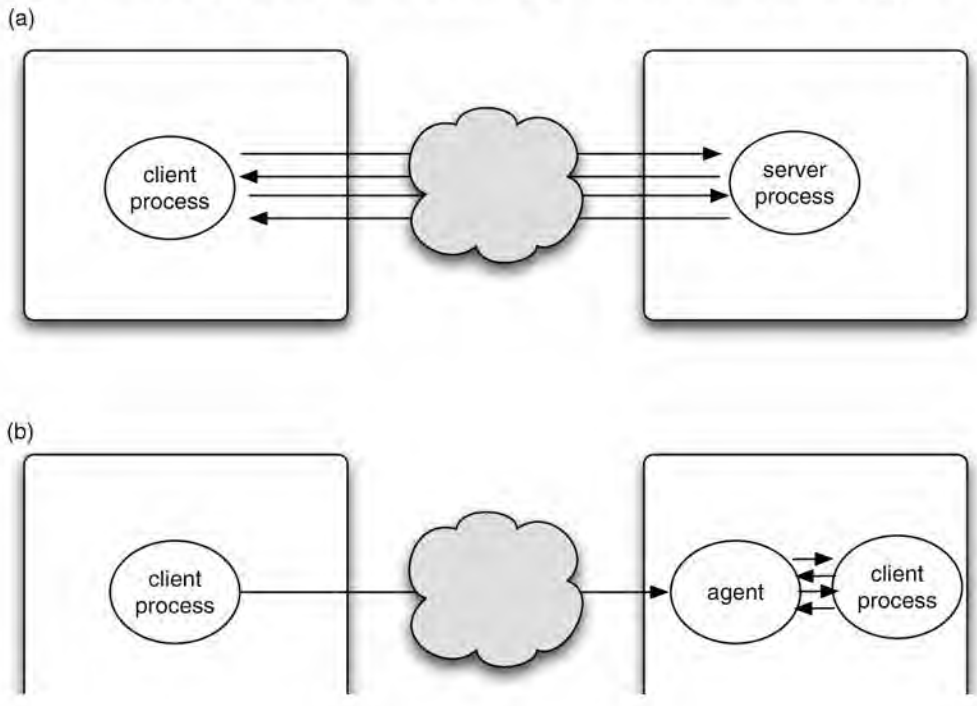
9.4 Mobile Agents

So far in this book I have avoided mention of an entire species of agent, which has aroused much interest, particularly in the programming-language and object-orienteddevelopment community. *Mobile* agents are agents that are capable of transmitting themselves – their program *and* their state – across a computer network, and recommencing execution at a remote site. Mobile agents became known largely through the pioneering work of General Magic, Inc., on their Telescript programming language, although there are now mobile agent platforms available for many languages and platforms (see Appendix A for some notes on the history of mobile agents).

The original motivation behind mobile agents is simple enough. The idea was that mobile agents would replace *remote procedure calls* as a way for processes to communicate over a network – see Figure 9.1. With remote procedure calls, the idea is that one process can invoke a procedure (method) on another process which is remotely located. Suppose one process `A` invokes a method `m` on process `B` with arguments `args;` the value returned by process `B` is to be assigned to a variable `v`. Using a Java-like notation, `A` executes an instruction somewhat like the following:

```
REMOTE PROCEDURE CALL
```

Figure 9.1: Remote procedure calls (a) versus mobile agents (b).



```
v=B.m(args)
```

Crucially, in remote procedure calls, communication is *synchronous*. That is, process A *blocks* from the time that it starts executing the instruction until the time that `B` returns a value. If `B` *never* returns a value – because the network fails, for example – then `A` may remain indefinitely suspended, waiting for a reply that will never come. The network connection between `A` and `B` may well also remain open, and even though it is largely unused (no data is being sent for most of the time), this may be costly.

BLOCKING COMMUNICATION

The idea of mobile agents (Figure 9.1(b)) is to replace the remote procedure call by sending out an agent to do the computation. Thus instead of invoking a method, process `A` sends out a program – a *mobile agent* – to process `B`. This program then interacts with process `B`. Since the agent shares the same address space as `B`, these interactions can be carried out much more efficiently than if the same interactions were carried out over a network. When the agent has completed its interactions, it returns to `A` with the required result. During the entire operation, the only network time required is that to send the agent to `B`, and that required to return the agent to `A` when it has completed its task. This is potentially a much more efficient use of network resources than the remote procedure call alternative described above. One of the original visions for Telescript was that it might provide an efficient way of managing network resources on devices such as handheld/palmtop computers, which might be equipped with expensive, limited-bandwidth Internet connections.

MOBILE AGENT

There are a number of technical issues that arise when considering mobile agents.

Serialization How is the agent serialized (i.e. encoded in a form suitable to be sent across the network), and, in particular, what aspects of the agent are serialized – the program, the data, or the program and its data?

Hosting and remote execution When the agent arrives at its destination, how is it executed, for example if the original host of the agent employs a different operating system or processor from the destination host?

Security When the agent from `A` is sent to the computer that hosts process `B`, there is obvious potential for the agent to cause trouble. It could potentially do this in a number of ways:

- it might obtain sensitive information by reading filestore or RAM directly
- it might deny service to other processes on the host machine, by either occupying too much of the available processing resource (processor cycles or memory) or else by causing the host machine to malfunction (for example by writing over the machine's RAM)
- it might simply cause irritation and annoyance, for example by causing windows to pop up on the user's GUI.

Many different answers have been developed to address these issues. With respect to the first issue – that of how to serialize and transmit an agent – there are several possibilities.

- Both the agent and its state are transmitted, and the state includes the program counter, i.e. the agent 'remembers' where it was before it was transmitted across the network, and when it reaches its destination, it recommences execution at the program instruction following that which caused it to be transmitted. This is the kind of mobility employed in the Telescript language [White, 1997].
- The agent contains both a program and the values of variables, but not the 'program counter', so the agent can remember the values of all variables, but not where it was when it transmitted itself across the network. This is how Danny Lange's Java-based Aglets framework works [Lange and Oshima, 1999].
- The agent to be transmitted is essentially a script, without any associated state (although state might be downloaded from the original host once the agent has arrived at its destination).

The issue of security dominates discussions about mobile agents. The key difficulty is that, in order for an agent to be able to do anything useful when it arrives at a remote location, it must access and make use of the resources supplied by the remote host. But providing access to these resources is inherently dangerous: it lays open the possibility that the host will be abused in some way. Languages like Java go some way to addressing these issues. For example, unlike languages such as C or C++, the Java language does not have pointers. It is thus inherently difficult (though not impossible) for a Java process to access the memory of the machine on which it is running. Java virtual machines also have a built-in `SecurityManager`, which defines the extent to which processes running on the virtual machine can access various resources. However, it is very hard to ensure that (for example) a process does not use more than a certain number of processor cycles.

Telescript

Telescript was a language-based environment for constructing multiagent systems developed in the early 1990s by General Magic, Inc. It was a commercial product, developed with the then very new hand-held computing market in mind [White, 1997].

There are two key concepts in Telescript technology: places and agents. Places are virtual locations that are occupied by agents – a place may correspond to a single machine, or a family of machines. Agents are the providers and consumers of goods in the *electronic marketplace* applications that Telescript was developed to support. Agents in Telescript are interpreted programs; the idea is rather similar to the way that Java bytecodes are interpreted by the Java virtual machine.

TELESCRIPT PLACES

Telescript agents are able to move from one place to another, in which case their program and state are encoded and transmitted across a network to another place, where execution recommences. In order to travel across the network, an agent uses a *ticket*, which specifies the parameters of its journey:

TELESCRIPT AGENTS

- the agent's destination
- the time at which the journey will be completed.

Telescript agents communicate with one another in several different ways:

- if they occupy different places, then they can connect across a network
- if they occupy the same location, then they can *meet* one another.

Telescript agents have an associated *permit*, which specifies what the agent can do (e.g. limitations on travel), and what resources the agent can use. The most important resources are

- 'money', measured in 'teleclicks' (which correspond to real money)
- lifetime (measured in seconds)
- size (measured in bytes).

Both Telescript agents and places are executed by an *engine*, which is essentially a virtual machine in the style of Java. Just as operating systems can limit the access provided to a process (e.g. in Unix, via access rights), so an engine limits the way an agent can access its environment. Engines continually monitor agents' resource consumption, and kill agents that exceed their limit. In addition, engines provide (C/C++) links to other applications via APIs.

TELESCRIPT PERMIT

Agents and places are programmed using the Telescript language. The Telescript language has the following characteristics.

- It is a pure object-oriented language – everything is an object (somewhat based on Smalltalk).
- It is interpreted, rather than compiled.
- It comes in two levels – *high* (the 'visible' language for programmers) and *low* (a semi-compiled language for efficient execution, rather like Java bytecodes).
- It contains a 'process' class, of which 'agent' and 'place' are subclasses.
- It is persistent, meaning that, for example, if a host computer was switched off and then on again, the state of the Telescript processes running on the host would have been automatically recorded, and execution would recommence automatically.

As noted in Appendix A, although Telescript was a pioneering language, which attracted a lot of attention, it was rapidly overtaken by Java, and throughout the late 1990s, a number of Java-based mobile agent frameworks appeared. The best known of these was Danny Lange's Aglets system.

Aglets – mobile agents in Java

Aglets is probably the best-known Java-based mobile agent platform. The core of Aglets lies in Java's ability to dynamically load and make instances of classes at run-time. An Aglet is an instance of a Java class that extends the `Aglet` class. When implementing such a class, the user can override a number of important methods provided by the `Aglet` class. The most important of these are

- the `onCreation ()` method, which allows an Aglet to initialize itself

- the `run ()` method, which is executed when an Aglet arrives at a new destination.

The core of an Aglet – the bit that does the work – is the `run ()` method. This defines the behaviour of the Aglet. Inside a `run ()` method, an Aglet can execute the `dispatch()` method, in order to transmit itself to a remote destination. An example of the use of the `dispatch()` method might be:

```
this.dispatch(new URL("atp://some.host.com/context1"));
```

This instruction causes the Aglet executing it to be serialized (i.e. its state to be recorded), and then sent to the 'context' called `context1` on the host `some. host. com`. A context plays the role of a host in Telescript; a single host machine can support many different Aglet contexts. In this instruction, `atp` is the name of the protocol via which the agent is transferred (in fact, `atp` stands for agent transfer protocol). When the agent is received at the remote host, an instance of the agent is created, the agent is initialized, its state is reconstructed from the serialized state sent with the agent, and, finally, the `run ()` method is invoked. Notice that this is *not* the same as Telescript, where the agent recommences execution at the program instruction following the `go` instruction that caused the agent to be transmitted. This information is lost in Aglets (although the user can record this information 'manually' in the state of the Aglet if required).

Agent Tcl and other scripting languages

The tool control language (Tcl – pronounced 'tickle') and its companion Tk are sometimes mentioned in connection with mobile agent systems. Tcl was primarily intended as a standard *command language* [Ousterhout, 1994]. The idea is that many applications (databases, spreadsheets, etc.) provide control languages, but every time a new application is developed, a new command language must be as well. Tcl provides the facilities to easily implement your own command language. Tk is an X-Window-based widget toolkit – it provides facilities for making GUI features such as buttons, labels, text and graphic windows (much like other X widget sets). Tk also provides powerful facilities for interprocess communication, via the exchange of Tcl scripts. Tcl/Tk combined make an attractive and simple-to-use GUI development tool; however, they have features that make them much more interesting:

- Tcl is an *interpreted language*
- Tcl is *extendable* – it provides a core set of primitives, implemented in C/C++, and allows the user to build on these as required
- Tcl/Tk can be *embedded* – the interpreter itself is available as C++ code, which can be embedded in an application, and can itself be extended.

Tcl programs are called *scripts*. These scripts have many of the properties that Unix shell scripts have:

- they are plain text programs that contain control structures (iteration, sequence, selection) and data structures (e.g. variables, lists, and arrays) just like a normal programming language
- they can be executed by a shell program (`tclsh` or `wish`)
- they can call up various other programs and obtain results from these programs (cf. procedure calls).

As Tcl programs are interpreted, they are very much easier to prototype and debug than compiled languages like C/C++ – they also provide more powerful control constructs. The idea of a mobile agent comes in because it is easy to build applications where Tcl scripts are exchanged across a network, and executed on remote machines. The *Safe Tcl* language provides mechanisms for limiting the access provided to a script. As an example, Safe Tcl controls the access that a script has to the GUI, by placing limits on the number of times a window can be modified by a script.

SAFE TCL

In summary, Tcl and Tk provide a rich environment for building language-based applications, particularly GUI-based ones. But they are not/were not intended as agent programming environments. The core primitives may be used for building agent programming environments – the source code is free, stable, well designed, and easily modified. The Agent Tcl framework is one attempt to do this [Gray, 1996; Kotz et al., 1997].

Notes and Further Reading

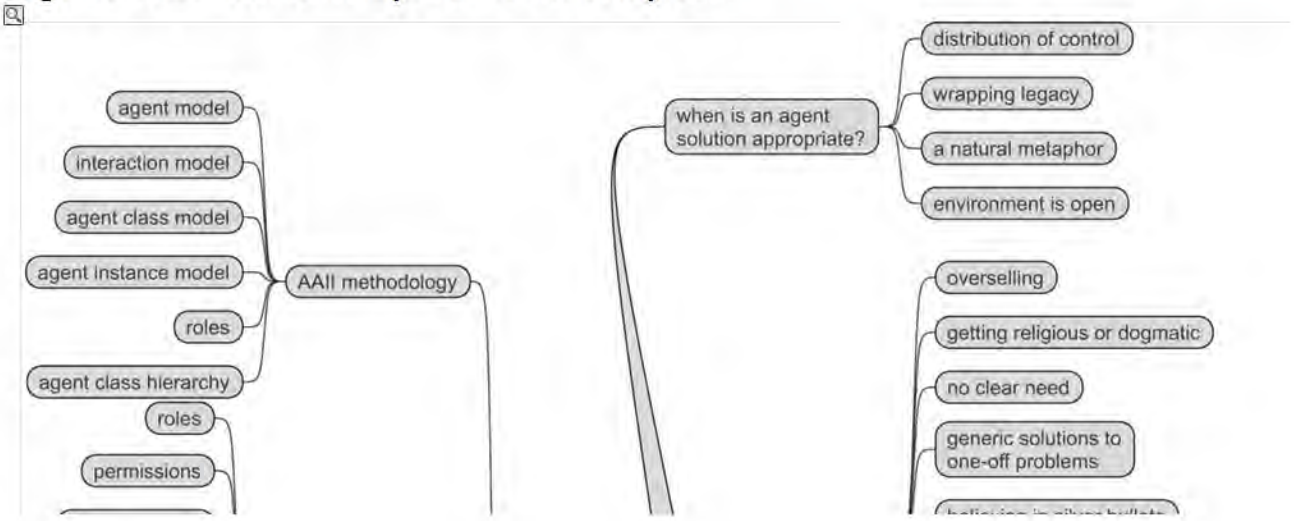
The area of methodologies for agent systems is an emerging one, and there is no obviously dominant approach in the literature. Detailed contemporary surveys of methodologies for agentoriented software engineering can be found in [Bergenti et al., 2004; Henderson-Sellers and Giorgini, 2005].

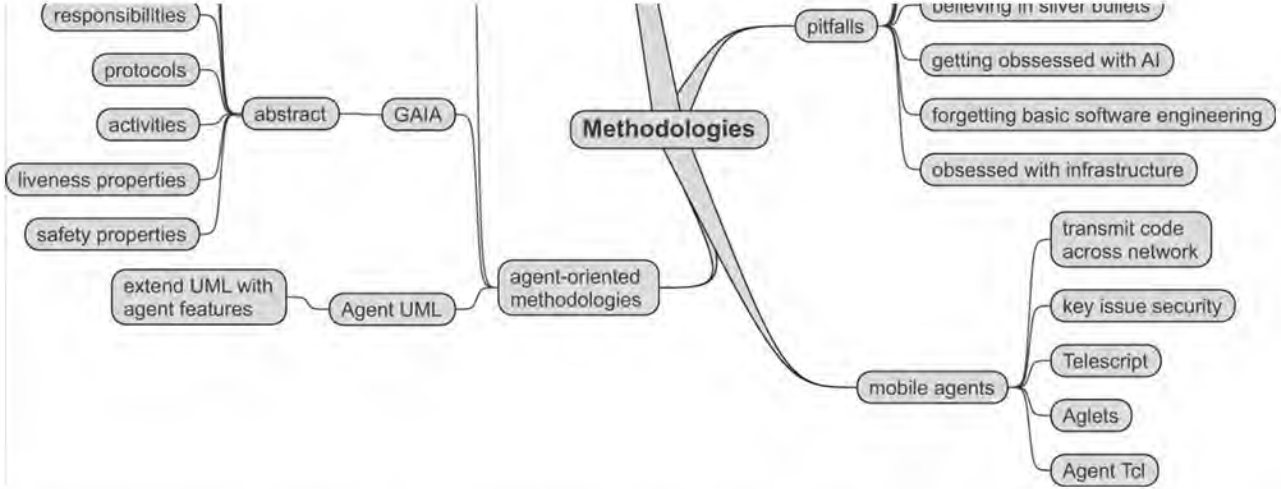
See [Wooldridge and Jennings, 1998] for a more detailed discussion of the pitfalls that await the agent developer, and [Webster, 1995] for the book that inspired this article.

There is a substantial literature on mobile agents; see, for example, [Rothermel and Popescu-Zeletin, 1997] for a collection of papers on the subject; also worth looking at are [Breugst et al., 1999; Brewington et al., 1999; Kiniry and Zimmerman, 1997; Knabe, 1995; Merz et al., 1997; Oshuga et al., 1997; Pham and Karmouch, 1998]. Security for mobile agent systems is discussed in [Tschudin, 1999; Yoshioka et al., 2001].

Class reading: [Kinny and Georgeff, 1997]. This article describes arguably the first agent-specific methodology. For a class familiar with OO methodologies, it may be worth discussing the similarities, differences, and what changes might be required to make this methodology really usable in practice.

Figure 9.2: Mind map for this chapter.





[1]The name comes from the Gaia hypothesis put forward by James Lovelock, to the effect that all the organisms in the Earth's biosphere can be viewed as acting together to regulate the Earth's environment.

Chapter 10 Applications

Agents have found application in many domains. We have seen in passing many of these mentioned so far. In this chapter, we will see some of the most notable application areas in more detail. Broadly speaking, applications of agents can be divided into two main groups:

Distributed systems in which agents become processing nodes in a distributed system. The emphasis in such systems is on the 'multi' aspect of multiagent systems.

Personal software assistants in which agents play the role of proactive assistants to users working with some application. The emphasis here is on 'individual' agents.

10.1 Agents for Workflow and Business Process Management

Workflow and business process control systems is an area of increasing importance in computer science. Workflow systems aim to automate the processes of a business, ensuring that different business tasks are expedited by the appropriate people at the right time, typically ensuring that a particular document flow is maintained and managed within an organization. The ADEPT system is a current example of an agent-based business process management system [Jennings et al., 1996b]. In ADEPT, a business organization is modelled as a society of negotiating, service-providing agents.

More specifically, the process was providing customers with a quote for installing a network to deliver a particular type of telecommunications service. This activity involves the following British Telecom (BT) departments: the Customer Service Division (CSD), the Design Division (DD), the Surveyor Department (SD), the Legal Division (LD), and the various organizations which provide the outsourced service of vetting customers (VCs). The process is initiated by a customer contacting the CSD with a set of requirements. In parallel to capturing the requirements, the CSD gets the customer vetted. If the customer fails the vetting procedure, the quote process terminates. Assuming the customer is satisfactory, their requirements are mapped against the service portfolio. If they can be met by an off-the-shelf item, then an immediate quote can be offered. In the case of bespoke services, however, the process is more complex. CSD further analyses the customer's requirements and, while this is occurring, LD checks the legality of the proposed service. If the desired service is illegal, the quote process terminates. If the requested service is legal, the design phase can start. To prepare a network design it is usually necessary to dispatch a surveyor to the customer's premises so that a detailed plan of the existing equipment can be produced. On completion of the network design and costing, DD informs CSD of the quote. CSD, in turn, informs the customer. The business process then terminates.

From this high-level system description, a number of autonomous problem-solving entities were identified. Thus, each department became an agent, and each individual within a department became an agent. To achieve their individual objectives, agents needed to interact with one another. In this case, all interactions took the form of negotiations about which services the agents would provide to one another and under what terms and conditions. The nature of these negotiations varied, depending on the context and the prevailing circumstances: interactions between BT internal agents were more cooperative than those involving external organizations, and negotiations where time is plentiful differed from those where time is short. In this context, negotiation involved generating a series of proposals and counter-proposals. If negotiation was successful, it resulted in a mutually agreeable contract. The agents were

negotiation was successful, it resulted in a mutually agreeable contract. The agents were arranged in various organizational structures: collections of agents were grouped together as a single conceptual unit (e.g. the individual designers and lawyers in DD and LD, respectively), authority relationships (e.g. the DD agent is the manager of the SD agent), peers within the same organization (e.g. the CSD, LD, and DD agents) and customer–subcontractor relationships (e.g. the CSD agent and the various VCs).

The ADEPT application had a clear rationale for adopting an agent-based solution. Centralized workflow systems are simply too unresponsive and are unable to cope with unpredictable events. It was decided, therefore, to devolve responsibility for managing the business process to software entities that could respond more rapidly to changing circumstances. Since there will inevitably be interdependencies between the various devolved functions, these software entities must interact to resolve their conflicts. Such a method of approach leaves autonomous agents as the most natural means of modelling the solution. Further arguments in favour of an agent-based solution are that agents provide a software model that is ideally suited to the devolved nature of the proposed business management system. Thus, the project's goal was to devise an agent framework that could be used to build agents for business process management. Note that ADEPT was neither conceived nor implemented as a general-purpose agent framework.

Rather than reimplementing communications from first principles, ADEPT was built on top of a commercial CORBA platform [OMG, 2001]. This platform provided the basis of handling distribution and heterogeneity in the ADEPT system. ADEPT agents also required the ability to undertake context-dependent reasoning and so a widely used expert system shell was incorporated into the agent architecture for this purpose. Development of either of these components from scratch would have consumed large amounts of project resources and would probably have resulted in a less robust and reliable solution. On the negative side, ADEPT failed to exploit any of the available standards for agent communication languages. This is a shortcoming that restricts the interoperation of the ADEPT system. In the same way that ADEPT exploited an off-the-shelf communications framework, so it used an architecture that had been developed in two previous projects (GRATE* [Jennings, 1993b] and ARCHON [Jennings et al., 1996a]). This meant the analysis and design phases could be shortened since an architecture (together with its specification) had already been devised.

The business process domain has a large number of legacy components (especially databases and scheduling software). In this case, these were generally wrapped up as resources or tasks within particular agents.

ADEPT agents embodied comparatively small amounts of AI technology. For example, planning was handled by having partial plans stored in a plan library (in the style of the Procedural Reasoning System [Georgeff and Lansky, 1987]). The main areas in which AI techniques were used was in the way agents negotiated with one another and the way that agents responded to their environment. In the former case, each agent had a rich set of rules governing which negotiation strategy it should adopt in which circumstances, how it should respond to incoming negotiation proposals, and when it should change its negotiation strategy. In the latter case, agents were required to respond to unanticipated events in a dynamic and uncertain environment. To achieve their goals in such circumstances they needed to be flexible about their individual and their social behaviour.

ADEPT agents were relatively coarse grained in nature. They represented organizations,

departments or individuals. Each such agent had a number of resources under its control, and was capable of a range of problem-solving behaviours. This led to a system design in which there were typically less than 10 agents at each level of abstraction and in which primitive agents were still capable of fulfilling some high-level goals.

10.2 Agents for Distributed Sensing

The classic application of multiagent technology was in distributed sensing [Durfee, 1988; Lesser and Erman, 1980]. The broad idea is to use multiagent systems to manage networks of spatially distributed sensors. The sensors may, for example, be acoustic sensors on a battlefield, or radars distributed across some airspace. The global goal of the system is to monitor and track all vehicles that pass within range of the sensors. The problem is that the sensors will typically provide partial and frequently conflicting data: different parts of the environment will have different characteristics with respect to sound and electromagnetic sensing spectrum, and different sensors will have different characteristics with respect to the quality of information they provide.

DISTRIBUTED SENSING

This task can be made simpler if the sensor nodes in the network *cooperate* with one another, for example by exchanging predictions about when a vehicle will pass from the region of one sensor to the region of another. This apparently simple domain has yielded surprising richness as an environment for experimentation into multiagent systems: Lesser's well-known *distributed vehicle monitoring testbed* (DVMT) provided the proving ground for many of today's multiagent system development techniques [Lesser and Erman, 1980].

DVMT

Figure 10.1: A typical sensing scenario for the DVMT.

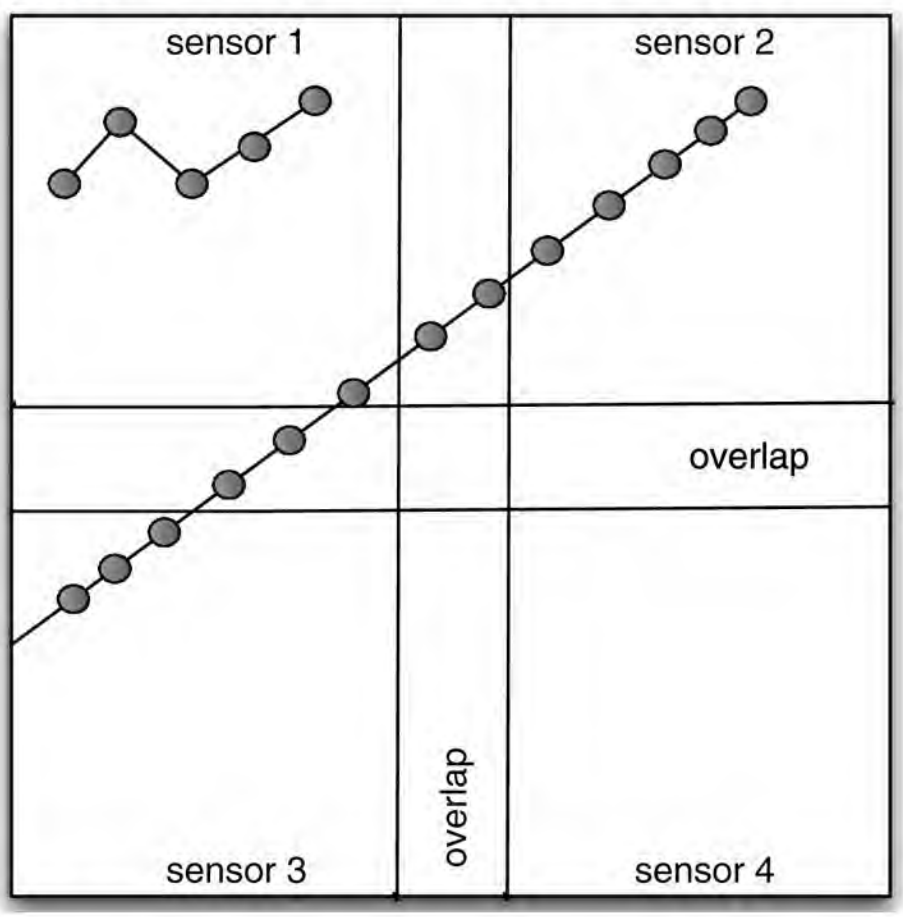


Figure 10.1 (from [Durfee, 1988, p. 136]) illustrates a typical DVMT situation. Here, two vehicles pass through the sensing space of four sensors, which have overlapping domains. In the figure, a dot represents a vehicle being sensed at a particular time. In the long track, the vehicle heads north east, starting by passing through the domain of sensor 3, then into shared sensor 1 and 3 space, then into sensor 1 space, then shared sensor 1 and 2 space, and finally into sensor 2 space. In the DVMT, an agent is associated with each sensor. It is easy for us to see that the 13 sensed points in this track are likely to be a single vehicle; the aim is to automate this kind of recognition. To do this in the DVMT requires the three relevant sensor agents in this scenario to cooperate in order to correctly interpret the data. For example, as the long vehicle track is about to leave sensor 1's domain, it can predict that the vehicle will enter the domain of sensor 2 and communicate the relevant data about this vehicle to sensor 2. While the vehicle is in space that overlaps the sensors, sensors 1 and 2 can cooperate by exchanging sensor data, perhaps allowing a more accurate track to be determined.

10.3 Agents for Information Retrieval and Management

The widespread provision of distributed, semi-structured information resources such as the Web obviously presents enormous potential; but it also presents a number of difficulties (such as 'information overload'). Agents have widely been proposed as a solution to these problems. An *information agent* is an agent that has access to at least one and potentially many information sources, and is able to collate and manipulate information obtained from these sources in order to answer queries posed by users and other information agents (the network of interoperating information sources are often referred to as intelligent and cooperative information systems [Papazoglou et al., 1992]). The information sources may be of many types, including, for example, traditional databases, as well as other information agents. Finding a solution to a query might involve an agent accessing information sources over a network. A typical scenario is that of a user who has heard about somebody at Stanford who has proposed something called agent-oriented programming. The agent is asked to investigate, and, after a careful search of various websites, returns with an appropriate technical report, as well as the name and contact details of the researcher involved.

INFORMATION AGENT

To see how agents can help in this task, consider the Web. What makes the Web so effective is that

- it allows access to networked, widely distributed information resources
- it provides a uniform interface to multimedia resources including text, images, sound, video, and so on
- it is hypertext based, making it possible to link documents together in novel or interesting ways
- perhaps most importantly, it has an extraordinarily simple and intuitive user interface, which can be understood and used in seconds.

The reality of Web use at the beginning of the 21st century is, however, still somewhat beset by problems. These problems may be divided into two categories: *human* and *organizational*.

Human factors

Human factors

The most obvious difficulty from the point of view of human users of the Web is the 'information overload' problem [Maes, 1994a]. People get overwhelmed by the sheer amount of information available, making it hard for them to filter out the junk and irrelevancies and focus on what is important, and also to actively search for the right information. Search engines such as Google and Yahoo attempt to alleviate this problem by indexing largely unstructured and unmanaged information on the Web. While these tools are useful, they tend to lack functionality: most search engines provide only simple search features, not tailored to a user's particular demands. In addition, current search engine functionality is directed at textual (typically HTML) content – despite the fact that one of the main selling features of the Web is its support for heterogeneous, multimedia content. Finally, it is not at all certain that the brute-force indexing techniques used by current search engines will scale to the size of the Internet in the next century. So *finding* and *managing* information on the Internet is, despite tools such as Google, still a problem.

In addition, people easily get bored or confused while browsing the Web. The hypertext nature of the Web, while making it easy to link related documents together, can also be disorienting – the 'back' and 'forward' buttons provided by most browsers are better suited to linear structures than the highly connected graph-like structures that underpin the Web. This can make it hard to understand the topology of a collection of linked web pages; indeed, such structures are inherently difficult for humans to visualize and comprehend. In short, it is all too easy to become lost in cyberspace. When searching for a particular item of information, it is also easy for people to either miss or misunderstand things.

Finally, the Web was not really designed to be used in a methodical way. Most web pages attempt to be attractive and highly animated, in the hope that people will find them interesting. But there is some tension between the goal of making a web page animated and diverting and the goal of conveying information. Of course, it *is* possible for a well-designed web page to effectively convey information, but, sadly, most web pages emphasize appearance, rather than content. It is telling that the process of using the web is known as 'browsing' rather than 'reading'. Browsing is a useful activity in many circumstances, but is not generally appropriate when attempting to answer a complex, important query.

Organizational factors

In addition, there are many organizational factors that make the Web difficult to use. Perhaps most importantly, apart from the (very broad) HTML standard, there are no standards for how a web page should look.

Another problem is the cost of providing online content. Unless significant information owners can see that they are making money from the provision of their content, they will simply cease to provide it. How this money is to be made is probably the dominant issue in the development of the Web today. I stress that these are not criticisms of the Web – its designers could hardly have anticipated the uses to which it would be put, nor that they were developing one of the most important computer systems to date. But these are all obstacles that need to be overcome if the potential of the Internet/Web is to be realized. The obvious question is then: what more do we need?

In order to realize the potential of the Internet, and overcome the limitations discussed above, it has been argued that we need tools that [Durfee et al., 1997]

- give a single coherent view of distributed, heterogeneous information resources
- give rich, *personalized*, user-oriented services, in order to overcome the 'information overload' problem – they must enable users to find the information they really want to find, and shield them from information they do not want
- are scalable, distributed, and modular, to support the expected growth of the Internet and the Web
- are adaptive and self-optimizing, to ensure that services are flexible and efficient.

Personal information agents

Many researchers have argued that agents provide such a tool. Pattie Maes from the MIT media lab is perhaps the best-known advocate of this work. She developed a number of prototypical systems that could carry out some of these tasks. I will here describe MAXIMS, an email assistant developed by Maes.

> [MAXIMS] learns to prioritize, delete, forward, sort, and archive mail messages on behalf of a user.
>
> [Maes, 1994a]

MAXIMS works by 'looking over the shoulder' of a user, and learning about how they deal with email. Each time a new event occurs (e.g. email arrives), MAXIMS records the event in the form of

$$\text{situation} \rightarrow \text{action}$$

pairs. A situation is characterized by the following attributes of an event:

- sender of email
- recipients
- subject line
- keywords in message body and so on.

When a new situation occurs, MAXIMS matches it against previously recorded rules. Using these rules, it then tries to predict what the user will do, and generates a *confidence level*: a real number indicating how confident the agent is in its decision. The confidence level is matched against two preset real number thresholds: a 'tell me' threshold and a 'do it' threshold. If the confidence of the agent in its decision is less than the 'tell me' threshold, then the agent gets feedback from the user on what to do. If the confidence of the agent in its decision is between the 'tell me' and 'do it' thresholds, then the agent makes a suggestion to the user about what to do. Finally, if the agent's confidence is greater than the 'do it' threshold, then the agent takes the initiative, and acts.

Rules can also be hard coded by users (e.g. 'always delete mails from person *X*'). MAX-IMS has a simple 'personality' (an animated face on the user's GUI), which communicates its 'mental state' to the user: thus the icon smiles when it has made a correct guess, frowns when it has made a mistake, and so on.

The NewT system is a Usenet news filter [Maes, 1994a, pp. 38–39]. A NewT agent is trained by giving it a series of examples, illustrating articles that the user would and would not choose to read. The agent then begins to make suggestions to the user, and is given feedback

choose to read. The agent then begins to make suggestions to the user, and is given feedback on its suggestions. NewT agents are not intended to remove human choice, but to represent an extension of the human's wishes: the aim is for the agent to be able to bring to the attention of the user articles of the type that the user has shown a consistent interest in. Similar ideas have been proposed by McGregor, who imagines *prescient agents* – intelligent administrative assistants, that predict our actions, and carry out routine or repetitive administrative procedures on our behalf [McGregor, 1992].

Web agents

[Etzioni and Weld, 1995] identify the following specific types of Web-based agent that they believe are likely to emerge in the near future.

> **Tour guides** The idea here is to have agents that help to answer the question 'where do I go next' when browsing the Web. Such agents can learn about the user's preferences in the same way that MAXIMS does, and, rather than just providing a single, uniform type of hyperlink, they actually indicate the likely interest of a link.
>
> **Indexing agents** Indexing agents will provide an extra layer of abstraction on top of the services provided by search/indexing tools such as Google. The idea is to use the raw information provided by such engines, together with knowledge of the user's goals, preferences, etc., to provide a *personalized* service.
>
> **FAQ-finders** The idea here is to direct users to 'frequently asked questions' (FAQs) documents in order to answer specific questions. Since FAQs tend to be knowledge-intensive, structured documents, there is a lot of potential for automated FAQ-finders.
>
> **Expertise finders**. Suppose I want to know about people interested in temporal belief logics. Current web search tools would simply take the three words 'temporal', 'belief' and 'logic', and search on them. This is not ideal: Google has no model of what you *mean* by this search, or what you really *want*. Expertise finders 'try to understand the user's wants and the contents of information services' in order to provide a better information provision service.

[Etzioni, 1996] put forward a model of information agents that *add value* to the underlying information infrastructure of the Web – the *information food chain* (see Figure 10.2). At the lowest level in the web information food chain is raw content: the home pages of individuals and companies. The next level up the food chain is the services that 'consume' this raw content. These services include search engines such as Google, Lycos, and Yahoo.

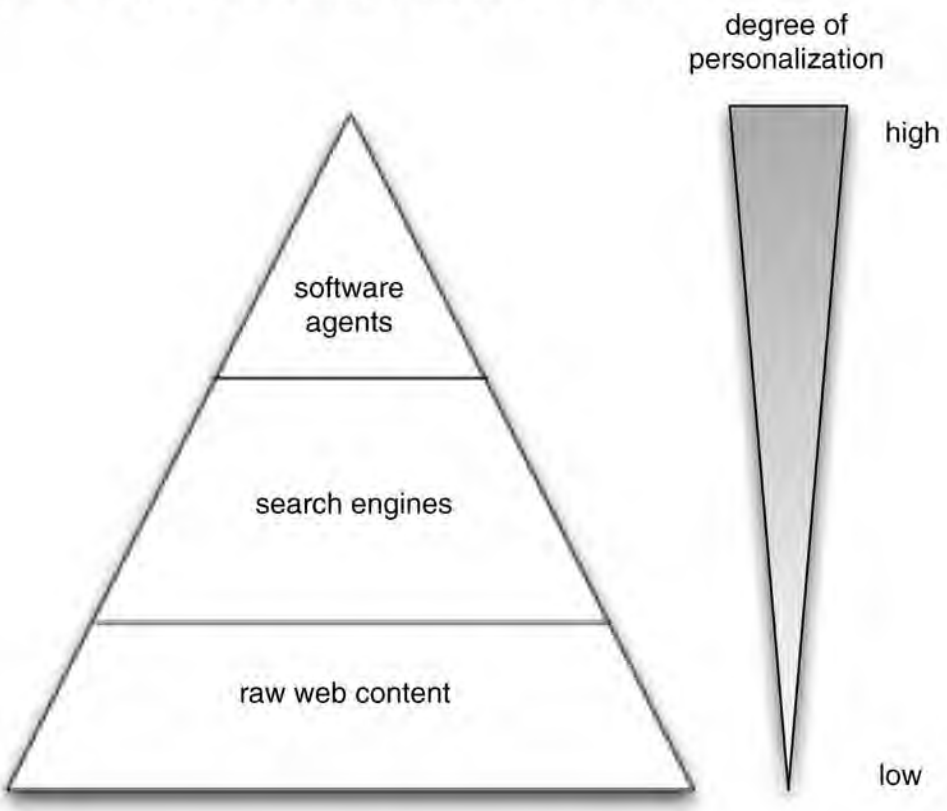
Search engines maintain large databases of web pages, indexed by content. Apart from the technical difficulties associated with storing such large databases and being able to process and retrieve their contents sufficiently quickly to provide a useful online service, the search engines must also *obtain* and *index* new or changed web pages on a regular basis. Currently, this is done in one of two ways. The simplest is to have humans search for pages and classify them manually. This has the advantage that the classifications obtained in this way are likely to be meaningful and useful. But it has the very obvious disadvantage that it is not necessarily thorough, and is costly in terms of human resources. The second approach is to use simple software agents, often called *spiders*, to systematically search the Web, following all links, and automatically classifying content. The classification of content is typically

all links, and automatically classifying content. The classification of content is typically done by removing 'noise' words from the page ('the', 'and', etc.), and then attempting to find those words that have the most meaning.

META SEARCH

All current search engines, however, suffer from the disadvantage that their coverage is partial. [Etzioni, 1996] suggested that one way around this is to use a *meta* search engine. This search engine works not by directly maintaining a database of pages, but by querying a number of search engines in parallel. The results from these search engines can then be collated and presented to the user. The meta search engine thus 'feeds' off the other search engines. By allowing the engine to run on the user's machine, it becomes possible to personalize services – to tailor them to the needs of individual users.

Figure 10.2: The web information food chain.



Multiagent information retrieval systems

The information resources – websites – in the kinds of applications I discussed above are essentially *passive*. They simply deliver specific pages when requested. A common approach is thus to make information resources more 'intelligent' by *wrapping* them with agent capabilities. The structure of such a system is illustrated in Figure 10.3.

AGENT WRAPPERS

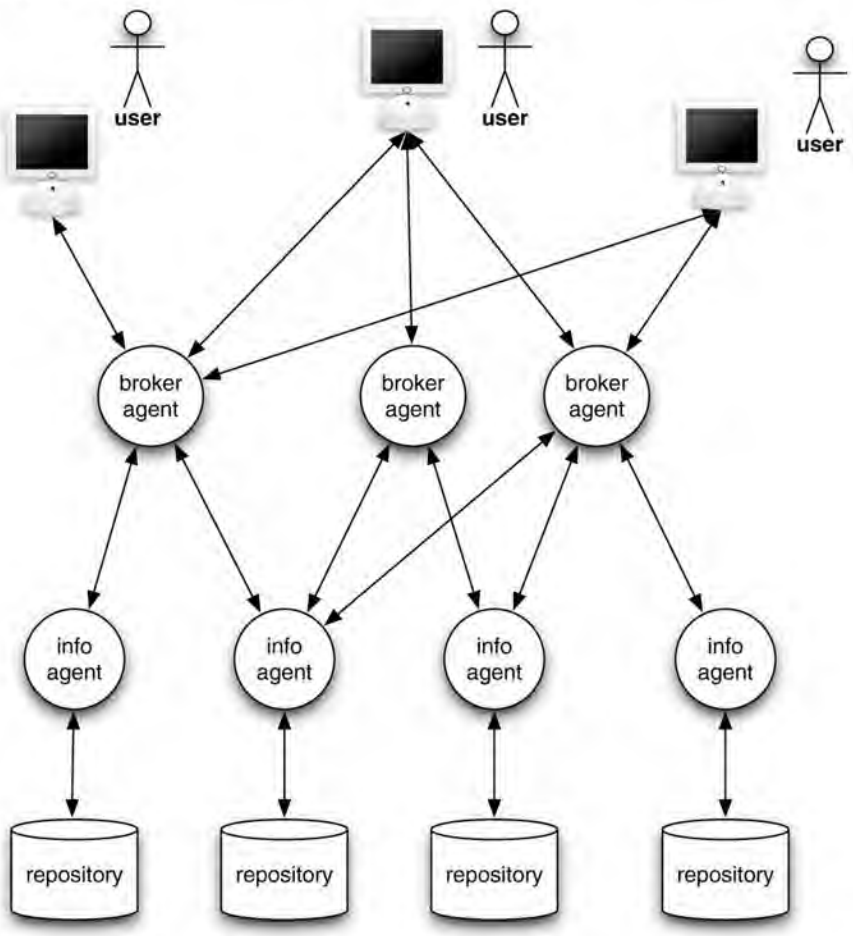
In this figure, there are a number of information repositories; these repositories may be websites, databases, or any other form of store. Access to these repositories is provided by information agents. These agents, which typically communicate using an agent communication language, are 'experts' about their associated repository. As well as being able to provide access to the repository, they are able to answer 'meta-level' queries about the content ('do you know about *X*?'). The agents will communicate with the repository using whatever native API the repository provides – HTTP, in the case of web repositories

using whatever native API the repository provides – HTTP, in the case of web repositories.

MIDDLE AGENTS

To address the issue of finding agents in an open environment like the Internet, *middle agents* or *brokers* are used [Kuokka and Harada, 1996; Wiederhold, 1992]. Each agent typically *advertises* its capabilities to some broker. Brokers come in several different types. They may be simply *matchmakers* or *yellow page* agents, which match advertisements to requests for advertised capabilities. Alternatively, they may be *blackboard* agents, which simply collect and make available requests. Or they may do both of these [Decker et al., 1997]. Different brokers may be specialized in different areas of expertise, for example knowing about different repositories.

Figure 10.3: Typical architecture of a multiagent information system.



Brokered systems are able to cope more quickly with a rapidly fluctuating agent population. Middle agents allow a system to operate robustly in the face of intermittent communications and agent appearance and disappearance.

The overall behaviour of a system such as that in Figure 10.3 is that a user issues a query to an agent on their local machine. This agent may then contact information agents directly, or it may go to a broker, which is skilled at the appropriate type of request. The broker may then contact a number of information agents, asking first whether they have the correct skills, and then issuing specific queries. This kind of approach has been successfully used in *digital library* applications [Wellman et al., 1996].

DIGITAL LIBRARY

10.4 Agents for Electronic Commerce

The boom in interest in the Internet throughout the late 1990s went hand-in-hand with an explosion of interest in electronic commerce (e-commerce) [Guilfoyle et al., 1997; Ovum, 1994]. As it currently stands, the Web has a number of features that limit its use as an 'information market'. Many of these stem from the fact that the Web has academic origins, and, as such, it was designed for free, open access. The Web was thus not designed to be used for commercial purposes, and a number of issues limit its use for this purpose.

Trust In an online global marketplace, it is difficult for consumers to know which vendors are reliable/secure and which are not, as they are faced with vendor brands that they have not previously encountered.

Privacy and security Consumers (still) have major worries about the security of their personal information when using e-commerce systems – mechanisms such as secure HTTP (`https`) go some way to alleviating this problem, but it remains a major issue.

Billing/revenue No built-in billing mechanisms are provided by the Web – they must be implemented over the basic Web structure; in addition, the Web was not designed with any particular revenue model in mind.

Reliability The Internet – and hence the Web – is unreliable, in that data and connections are frequently lost, and it thus has unpredictable performance.

'First-generation' e-commerce systems (of which `amazon.com` was perhaps the best known example) allowed a user to browse an online catalogue of products, choose some, and then purchase these selected products using a credit card. However, agents make *second-generation* e-commerce systems possible, in which many aspects of a consumer's buying behaviour are automated.

There are many models that attempt to describe consumer buying behaviour. Of these, one of the most popular postulates that consumers tend to engage in the following six steps [Guttman et al., 1998, pp. 148,149].

(1) Need identification This stage characterizes the consumer becoming aware of some need that is not satisfied.

(2) Product brokering In this stage, a would-be consumer obtains information relating to available products, in order to determine *what* product to buy.

(3) Merchant brokering In this stage, the consumer decides *who* to buy from. This stage will typically involve examining offers from a range of different merchants.

(4) Negotiation In this stage, the terms of the transaction are agreed between the would-be consumer and the would-be merchant. In some markets (e.g. regular retail markets), the negotiation stage is empty – the terms of agreement are fixed and not negotiable. In other markets (e.g. the used car market), the terms are negotiable.

(5) Purchase and delivery In this stage, the transaction is actually carried out, and the good delivered.

(6) Product service and evaluation The post-purchase stage involves product service, customer service, etc.

Agents have been widely promoted as being able to automate (or at least partially automate) some of these stages, and hence assist the consumer to reach the best deal possible [Noriega and Sierra, 1999].

Comparison shopping agents

The simplest type of agent for e-commerce is the *comparison shopping* agent. The idea is very similar to that of the meta search engines discussed above. Suppose you want to purchase the CD 'Music' by Madonna. Then a comparison shopping agent will search a number of online shops to find the best deal possible.

COMPARISON SHOPPING

Such agents work well when the agent is required to compare goods with respect to a single attribute – typically price. The obvious examples of such situations are 'shrink wrapped' goods such as CDs and books. However, they work less well when there is more than one attribute to consider. An example might be the used-car market, where, in addition to considering price, the putative consumer would want to consider the reputation of the merchant, the length and type of any guarantee, and many other attributes.

The Jango system [Doorenbos et al., 1997] is a good example of a first-generation e-commerce agent. The long-term goals of the Jango project were to

- help the user decide what to buy
- find specifications and reviews of products
- make recommendations to the user
- perform comparison shopping for the best buy
- monitor 'what's new' lists
- watch for special offers and discounts.

A key obstacle that the developers of Jango encountered was simply that web pages are all different. Jango exploited several *regularities* in vendor websites in order to make 'intelligent guesses' about their content.

> **Navigation regularity** Websites are designed by vendors so that products are easy to find.
>
> **Corporate regularity** Websites are usually designed so that pages have a similar 'look' n 'feel'
>
> **Vertical separation** Merchants use white space to separate products.

Internally, Jango has two key components:

- a component to *learn vendor descriptions* (i.e. learn about the structure of vendor web pages)
- a *comparison shopping* component, capable of comparing products across different vendor sites.

In 'second-generation' agent-mediated electronic commerce systems, it is proposed that

agents will be able to assist with the fourth stage of the purchasing model set out above: negotiation. The idea is that a would-be consumer delegates the authority to negotiate terms to a software agent. This agent then negotiates with another agent (which may be a software agent or a person) in order to reach an agreement.

There are many obvious hurdles to overcome with respect to this model: the most important of these is *trust*. Consumers will not delegate the authority to negotiate transactions to a software agent unless they trust the agent. In particular, they will need to trust that the agent (i) really understands what they want, and (ii) is not going to be exploited ('ripped off') by another agent, and end up with a poor agreement.

Comparison shopping agents are particularly interesting because it would seem that, if the user is able to search the entire marketplace for goods at the best price, then the overall effect is to force vendors to push prices as low as possible. Their profit margins are inevitably squeezed, because otherwise potential purchasers would go elsewhere to find their goods.

10.5 Agents for Human–Computer Interfaces

Currently, when we interact with a computer via a user interface, we are making use of an interaction paradigm known as *direct manipulation*. Put simply, this means that a computer program (a word processor, for example) will only do something if we explicitly tell it to, for example by clicking on an icon or selecting an item from a menu. When we work with humans on a task, however, the interaction is more two-way: we work with them as peers, each of us carrying out parts of the task and proactively helping each other as problem solving progresses. In essence, the idea behind interface agents is to make computer systems more like proactive assistants. Thus, the goal is to have computer programs that in certain circumstances could *take the initiative*, rather than wait for the user to spell out exactly what they wanted to do. This leads to the view of computer programs as *cooperating* with a user to achieve a task, rather than acting simply as servants. A program capable of taking the initiative in this way would in effect be operating as a semi-autonomous agent. Such agents are sometimes referred to as *expert assistants* or *interface agents*. [Maes, 1994b, p. 71] defines interface agents as

DIRECT MANIPULATION

INTERFACE AGENTS

> computer programs that employ artificial intelligence techniques in order to provide assistance to a user dealing with a particular application.... The metaphor is that of a *personal assistant* who is *collaborating with the user* in the same work environment.

One of the key figures in the development of agent-based interfaces has been Nicholas Negroponte (director of MIT's influential Media Lab). His vision of agents at the interface was set out in his 1995 book *Being Digital* [Negroponte, 1995]:

> The agent answers the phone, recognizes the callers, disturbs you when appropriate, and may even tell a white lie on your behalf. The same agent is well trained in timing, versed in finding opportune moments, and respectful of idiosyncrasies.... If you have somebody who knows you well and shares much of your information, that person can act on your behalf very effectively. If your secretary falls ill, it would make no difference if the temping agency could send you Albert Einstein. This issue is not about IQ. It is shared

> knowledge and the practice of using it in your best interests…. Like an army commander sending a scout ahead … you will dispatch agents to collect information on your behalf. Agents will dispatch agents. The process multiplies. But [this process] started at the interface where you delegated your desires.

10.6 Agents for Virtual Environments

There is obvious potential for marrying agent technology with that of the cinema, computer games, and virtual reality. The OZ project was initiated to develop:

> … artistically interesting, highly interactive, simulated worlds … to give users the experience of living in (not merely watching) dramatically rich worlds that include moderately competent, emotional agents.
>
> [Bates et al., 1992b, p. 1]

In order to construct such simulated worlds, one must first develop *believable agents:* agents that 'provide the illusion of life, thus permitting the audience's suspension of disbelief' [Bates, 1994, p. 122]. A key component of such agents is *emotion:* agents should not be represented in a computer game or animated film as the flat, featureless characters that appear in current computer games. They need to show emotions; to act and react in a way that resonates in tune with our empathy and understanding of human behaviour. The OZ group investigated various architectures for emotion [Bates et al., 1992a], and have developed at least one prototype implementation of their ideas [Bates, 1994].

BELIEVABLE AGENTS

10.7 Agents for Social Simulation

I noted in Chapter 1 that one of the visions behind multiagent systems technology is that of using agents as an experimental tool in the social sciences [Gilbert, 1995; Gilbert and Doran, 1994; Moss and Davidsson, 2001]. Put crudely, the idea is that agents can be used to simulate the behaviour of human societies. At its simplest, individual agents can be used to represent individual people; alternatively, individual agents can be used to represent organizations and similar such entities.

[Conte and Gilbert, 1995, p. 4] suggest that multiagent simulation of social processes can have the following benefits:

- computer simulation allows the observation of properties of a model that may *in principle* be analytically derivable but have not yet been established
- possible alternatives to a phenomenon observed in nature may be found
- properties that are difficult/awkward to observe in nature may be studied at leisure in isolation, recorded, and then 'replayed' if necessary
- 'sociality' can be modelled explicitly – agents can be built that have representations of other agents, and the properties and implications of these representations can be investigated.

[Moss and Davidsson, 2001, p. 1] succinctly states a case for multiagent simulation:

> [For many systems,] behaviour cannot be predicted by statistical or qualitative analysis.

> … Analysing and designing … such systems requires a different approach to software engineering and mechanism design.

Moss goes on to give a general critique of approaches that focus on formal analysis at the expense of accepting and attempting to deal with the 'messiness' that is inherent in most multiagent systems of any complexity. There is undoubtedly some strength to these arguments, which echo cautionary comments made by some of the most vocal proponents of game theory [Binmore, 1992, p. 196]. In the remainder of this section, I will review one major project in the area of social simulation, and point to some others.

The EOS project

The EOS project, undertaken at the University of Essex in the UK, is a good example of a social simulation system [Doran, 1987; Doran and Palmer, 1995; Doran et al., 1992]. The aim of the EOS project was to investigate the causes of the emergence of social complexity in Upper Palaeolithic France. Between 15 000 and 30 000 years ago, at the height of the last ice age, there was a relatively rapid growth in the complexity of societies that existed at this time. The evidence of this social complexity came in the form of [Doran and Palmer, 1995]:

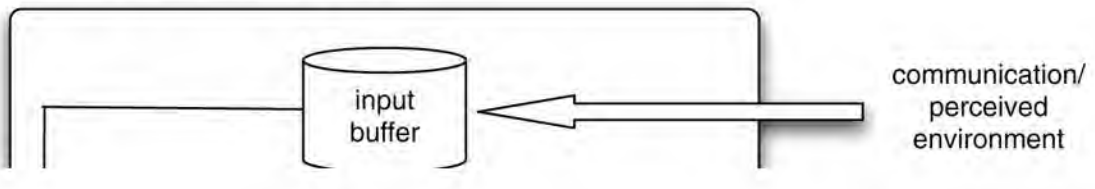
- larger and more abundant archaeological sites
- increased wealth, density, and stratigraphic complexity of archaeological material in sites
- more abundant and sophisticated cave art (the well-known caves at Lascaux are an example)
- increased stone, bone, and antler technology
- abundance of 'trade' objects.

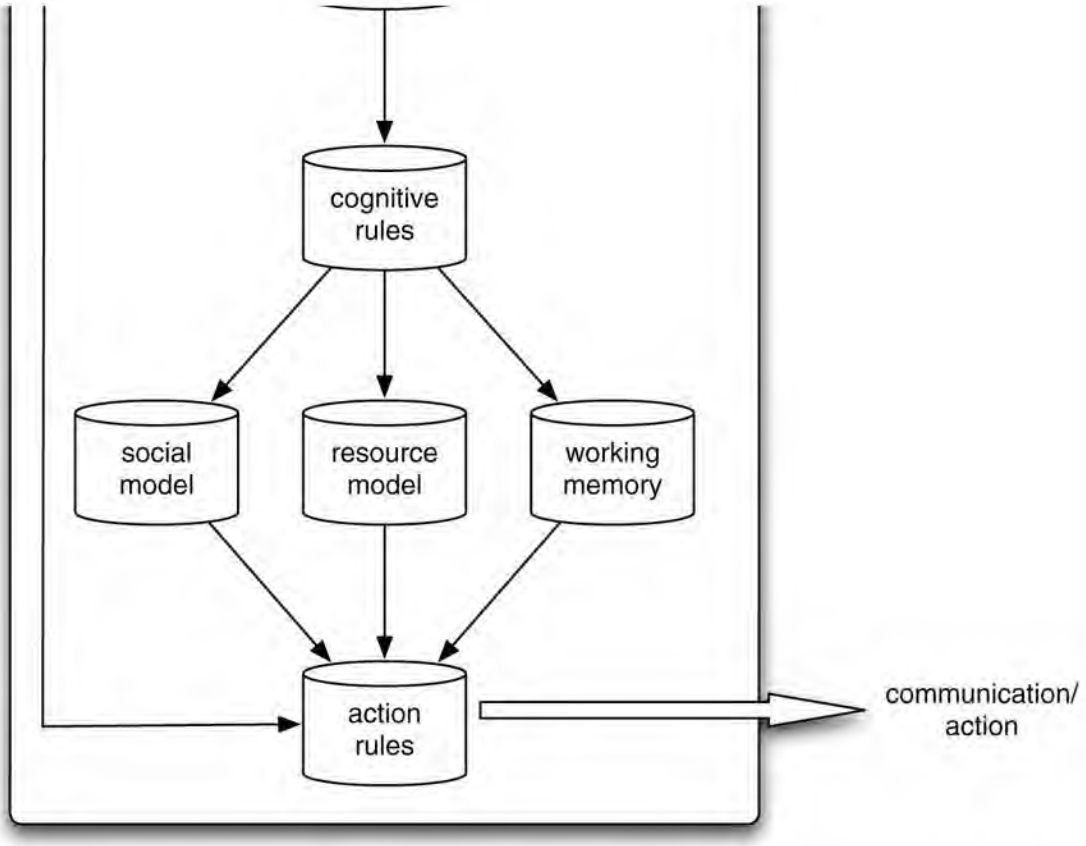
A key open question for archaeologists is what exactly *caused* this emergence of complexity. In 1985, the archaeologist Paul Mellars proposed a model that attempted to explain this complexity. The main points of Mellars' model were that the key factors leading to this growth in complexity were an exceptional wealth and diversity of food resources, and a strong, stable, predictable concentration of these resources.

In order to investigate this model, a multiagent experimental platform – the EOS testbed – was developed. This testbed, implemented in the Prolog language [Clocksin and Mellish, 1981], allows agents to be programmed as rule-based systems. The structure of an EOS agent is shown in Figure 10.4.

Each agent in EOS is endowed with a symbolic representation of its environment – its beliefs. Beliefs are composed of beliefs about other agents (the *social model*), beliefs about resources in the environment (the *resource model*), and miscellaneous other beliefs. To update its beliefs, an agent has a set of *cognitive rules*, which map old beliefs to new ones. To decide what action to perform, agents have *action rules*, which map beliefs to actions. (Compare with Shoham's AGENT0 system described in Chapter 3.) Both cognitive rules and action rules are executed in a forward-chaining manner.

Figure 10.4: Agents in EOS.





Agents in the EOS testbed inhabit a simulated two-dimensional environment, some 10 000 × 10 000 cells in size (cf. the Tileworld described in Chapter 2.) Each agent occupies a single cell, initially allocated at random. Agents have associated with them *skills*. The idea is that an agent will attempt to obtain resources ('food') which are situated in the environment, but resources come in different types, and only agents of certain types are able to obtain certain resources. Agents have a number of 'energy stores', and for each of these a 'hunger level'. If the energy store associated with a particular hunger level falls below the value of the hunger level, then the agent will attempt to replenish it by consuming appropriate resources. Agents travel about the EOS world in order to obtain resources, which are scattered about the world. Recall that the Mellars model suggested that the availability of resources at predictable locations and times was a key factor in the growth of the social complexity in the Palaeolithic period. To reflect this, resources (intuitively corresponding to things like a reindeer herd or a fruit tree) were clustered, and the rules governing the emergence and disappearance of resources reflects this.

The basic form of social structure that emerges in EOS does so because certain resources have associated with them a *skill profile*. This profile defines, for every type of skill or capability that agents may possess, how many agents with this skill are required to obtain the resource. For example, a 'fish' resource might require two 'boat' capabilities; and a 'deer' resource might require a single 'spear' capability.

In each experiment, a user may specify a number of parameters:

- the number of resource locations of each type and their distribution
- the number of resource instances that each resource location comprises
- the type of energy that each resource location can supply
- the quantity of energy an instance of a particular resource can supply
- the skill profiles for each resource

- the 'renewal' period, which elapses between a resource being consumed and being replaced.

To form collaborations in order to obtain resources, agents use a variation of Smith's Contract Net protocol (see Chapter 8). Thus, when an agent finds a resource, it can advertise this fact by sending out a broadcast announcement. Agents can then bid to collaborate on obtaining a resource, and the successful bidders then work together to obtain the resource.

A number of social phenomena were observed in running the EOS testbed, for example: 'overcrowding' when too many agents attempt to obtain resources in some locale; 'clobbering' when agents accidentally interfere with each other's goals; and semi-permanent groups arising. With respect to the emergence of deep hierarchies of agents, it was determined that the growth of hierarchies depended to a great extent on the perceptual capabilities of the group. If the group is not equipped with adequate perceptual ability, then there is insufficient information to cause a group to form. A second key aspect in the emergence of social structure is the complexity of resources – how many skills it requires in order to obtain and exploit a resource. If resources are too complex, then groups will not be able to form to exploit them before they expire.

An interesting aspect of the EOS project was that it highlighted the *cognitive* aspects of multiagent social simulation. That is, by using EOS, it was possible to see how the beliefs and aspirations of individuals in a society can influence the possible trajectories of this society. One of the arguments in favour of this style of multiagent societal simulation is that this kind of property is very hard to model or understand using analytical techniques such as game or economic theory (cf. the quote from Moss and Davidsson, above).

Policy modelling by multiagent simulation

Another application area for agents in social simulation is that of *policy modelling and development* [Downing et al., 2001]. Regulatory and other similar bodies put forward policies, which are designed – or at least intended – to have some desired effect.

An example might be related to the issue of potential climate change caused by the release of greenhouse gases (cf. [Downing et al., 2001]). A national government, or an international body such as the EU, might desire to reduce the potentially damaging effects of climate change, and put forward a policy designed to limit it. A typical first cut at such a policy might be to increase fuel taxes, the idea being that this reduces overall fuel consumption, in turn reducing the release of greenhouse gases. But policy makers must generally form their policies in ignorance of what the *actual* effect of their policies will be, and, in particular, the actual effect may be something quite different from that intended. In the greenhouse gas example, the effect of increasing fuel taxes might be to cause consumers to switch to cheaper – dirtier – fuel types, at best causing no overall reduction in the release of greenhouse gases, and potentially even leading to an increase. So, it is proposed that multiagent simulation models might fruitfully be used to gain an understanding of the effect of their nascent policies.

An example of such a system is the Freshwater Integrated Resource Management with Agents (FIRMA) project [Downing et al., 2001]. This project is specifically intended to understand the impact of governments exhorting water consumers to exercise care and caution in water use during times of drought [Downing et al., 2001, p. 206]. (In case you were wondering, yes, droughts *do* happen in the UK!) [Downing et al., 2001] developed a

were wondering, yes, droughts do happen in the UK!) [Downing et al., 2001] developed a multiagent simulation model in which water consumers were represented by agents, and a 'policy' agent issued exhortations to consume less at times of drought. The authors were able to develop a simulation model that fairly closely resembled the observed behaviour in human societies in similar circumstances; developing this model was an iterative process of model reformulation followed by a review of the observed results of the model with water utilities.

10.8 Agents for *X*

Agents have been proposed for many more application areas than I have the space to discuss here. In this section, I will give a flavour of some of these.

Agents for industrial systems management ARCHON was one of the largest and best-known European multiagent system development projects to date [Jennings et al., 1995; Jennings and Wittig, 1992; Wittig, 1992]. This project developed and deployed multiagent technology in several industrial domains. The most significant of these domains was a power distribution system, which was installed and is currently operational in northern Spain. Agents in ARCHON have two main parts: a *domain* component, which realizes the domain-specific functionality of the agent; and a *wrapper* component, which provides the agent functionality, enabling the system to plan its actions, and to represent and communicate with other agents. The ARCHON technology has subsequently been deployed in several other domains, including particle accelerator control. (ARCHON was the platform through which Jennings's joint intention model of cooperation [Jennings, 1995], discussed in Chapter 8, was developed.)

Agents for air-traffic control Air-traffic control systems are among the oldest application areas in multiagent systems [Findler and Lo, 1986; Steeb et al., 1988]. A recent example is OASIS (Optimal Aircraft Sequencing using Intelligent Scheduling), a system that is currently undergoing field trials at Sydney airport in Australia [Ljungberg and Lucas, 1992]. The specific aim of OASIS is to assist an air-traffic controller in managing the flow of aircraft at an airport: it offers estimates of aircraft arrival times, monitors aircraft progress against previously derived estimates, informs the air-traffic controller of any errors, and perhaps most importantly finds the optimal sequence in which to land aircraft. OASIS contains two types of agents: *global* agents, which perform generic domain functions (for example, there is a 'sequencer agent', which is responsible for arranging aircraft into a least-cost sequence); and *aircraft agents*, one for each aircraft in the system airspace. The OASIS system was implemented using the PRS agent architecture.

Notes and Further Reading

[Jennings and Wooldridge, 1998a] is a collection of papers on applications of agent systems. [Parunak, 1999] gives a more recent overview of industrial applications. [Hayzelden and Bigham, 1999] is a collection of articles loosely based around the theme of agents for computer network applications; [Klusch, 1999] is a similar collection centred around the topic of information agents. [Lesser et al., 2003] is a collection of articles on multiagent systems for distributed sensor networks.

[Van Dyke Parunak, 1987] describes the use of the Contract Net protocol for manufacturing control in the YAMS (Yet Another Manufacturing System). Mori et al. have used a

multiagent approach to controlling a steel coil processing plant [Mori et al., 1988].

A number of studies have been made of information agents, including a theoretical study of how agents are able to incorporate information from different sources [Gruber, 1991; Levy et al., 1994], as well as a prototype system called IRA (information retrieval agent) that is able to search for loosely specified articles from a range of document repositories [Voorhees, 1994]. Another important system in this area was Carnot [Huhns et al., 1992], which allows pre-existing and heterogeneous database systems to work together to answer queries that are outside the scope of any of the individual databases.

There is much related work being done by the computer-supported cooperative work (CSCW) community. CSCW is informally defined by Baecker to be 'computer assisted coordinated activity such as problem solving and communication carried out by a group of collaborating individuals' [Baecker, 1993, p. 1]. The primary emphasis of CSCW is on the development of (hardware and) software tools to support collaborative human work – the term *groupware* has been coined to describe such tools. Various authors have proposed the use of agent technology in groupware. For example, in his *participant systems* proposal, Chang suggests systems in which humans collaborate not only with other humans, but also with artificial agents [Chang, 1987]. We refer the interested reader to the collection of papers edited by [Baecker, 1993] and the article by [Greif, 1994] for more details on CSCW.

GROUPWARE

PARTICIPANT AGENTS

[Noriega and Sierra, 1999] is a collection of papers on agent-mediated electronic commerce. [Kephart and Greenwald, 1999] investigates the dynamics of systems in which buyers and sellers are agents.

[Gilbert and Conte, 1995; Gilbert and Doran, 1994; Moss and Davidsson, 2001] are collections of papers on the subject of simulating societies by means of multiagent systems. [Davidsson, 2001] discusses the relationship between multiagent simulation and other types of simulation (e.g. object-oriented simulation and discrete event models).

Class reading: [Parunak, 1999]. This paper gives an overview of the use of agents in industry from one of the pioneers of agent applications.

Part IV Multiagent Decision Making

We have now looked at decision-making architectures for building agents, and how such agents can communicate and cooperate to solve problems. However, we have not considered in much detail the particular aspects of decision-making in societies where agents are self-interested. In this part, we look at such decision-making. We focus on the issue of *reaching agreement*. We consider a number of different types of agreement:

- First, we introduce the basic concepts surrounding multiagent encounters: how to model decision settings, and the properties of good decisions in such settings. This leads us to the fundamental question of when cooperation is possible, and how cooperation can be facilitated.
- Second, we consider techniques by which a group of self-interested agents may select some outcome from a range of possibilities. The techniques we consider here are based on the theory of *social choice*, or *voting*.
- Third, we consider situations in which binding agreements may be made between agents, leading to the possibility of coalitions forming. We investigate the main techniques for modelling situations.
- Fourth, we consider the problem of allocating scarce resources in societies where agents value these resources differently: the techniques we study here are based on *auctions*.
- Fifth, we consider the possibility of using *bargaining* or *negotiation* to reach agreement; for example, we show how bargaining techniques can be used to reallocate tasks and resources to mutual benefit.
- Finally, we consider the problem of how to resolve conflicts over beliefs. We investigate the use of *argumentation* – rational discourse – as an approach to deriving a mutually acceptable position on some domain of discourse on which differing beliefs are held.

Chapter 11 Multiagent Interactions

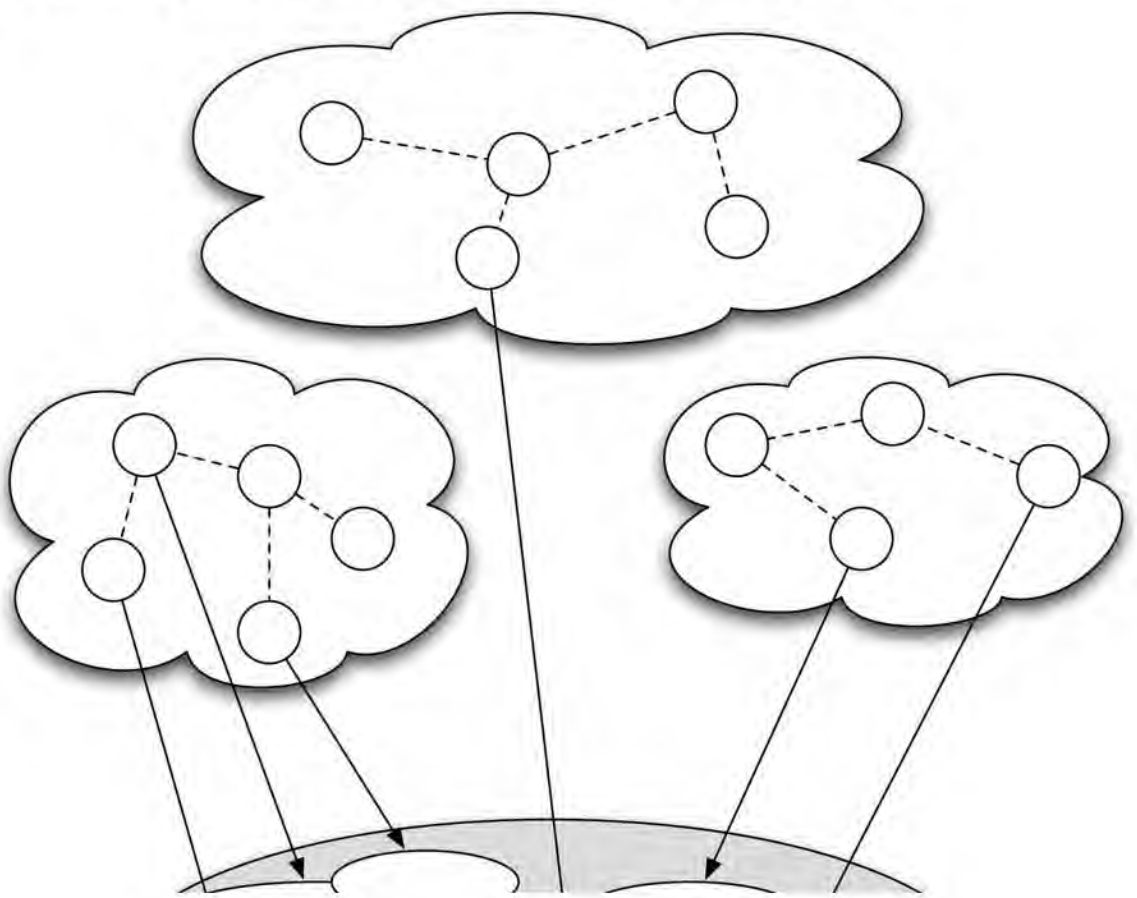
Figure 11.1 (from [Jennings, 2000]) illustrates the typical structure of a multiagent system. The system contains a number of agents, which communicate with one another. The agents are able to act in an environment; different agents have different 'spheres of influence', in the sense that they will have control over – or at least be able to influence – different parts of the environment. These spheres of influence may coincide in some cases. The fact that these spheres of influence may coincide may give rise to dependencies between the agents. For example, two robotic agents may both be able to move through a door – but they may not be able to do so simultaneously. Finally, agents will also typically be linked by other relationships. For example, there might be 'power' relationships, where one agent is subservient to another.

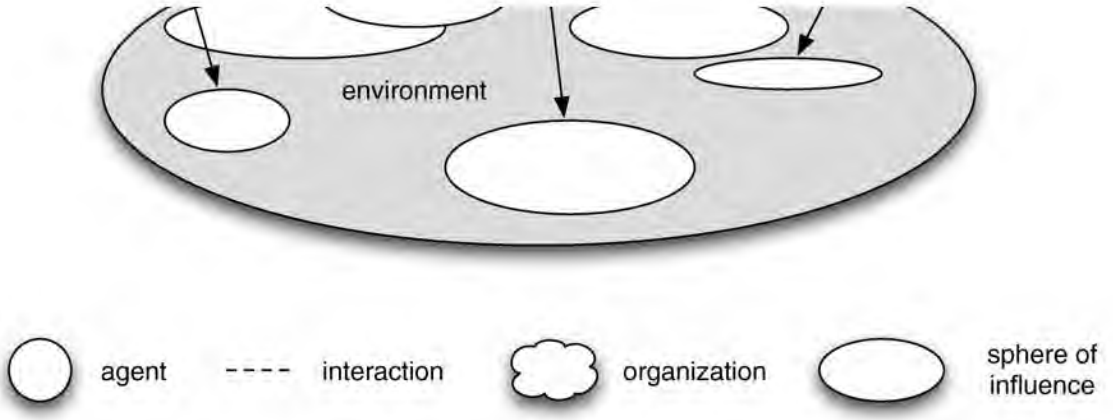
The most important lesson of this chapter – and perhaps one of the most important lessons of multiagent systems generally – is that, when faced with what appears to be a multiagent domain, it is critically important to understand the *type* of interaction that takes place between the agents in order to be able to make the best decision possible about what action to perform.

11.1 Utilities and Preferences

First, let us simplify things by assuming that we have just two agents; notation tends to be much messier when we have more than two. Fortunately, we need only two agents to illustrate the key ideas. Let us call these two agents i and j. Each agent is assumed to be *self-interested*. That is, each agent has its own preferences and desires about how the world should be. For the moment, we will not be concerned with where these preferences come from; just assume that they are the preferences of the agent's user or owner. Next, we will assume that there is a set $\Omega = \{\omega_1, \omega_2, \ldots\}$ of 'outcomes' that the agents have preferences over. To make this concrete, just think of these as outcomes of a game that the two agents are playing.

Figure 11.1: Typical structure of a multiagent system.





We formally capture the preferences that the two agents have by means of *utility functions*, one for each agent, which assign to every outcome a real number, indicating how 'good' the outcome is for that agent. The larger the number the better from the point of view of the agent with the utility function. Thus agent i's preferences will be captured by a function

UTILITIES

$$u_i : \Omega \to \mathbb{R}$$

and agent j's preferences will be captured by a function

$$u_j : \Omega \to \mathbb{R}.$$

(Compare with the discussion in Chapter 2 on tasks for agents.) It is not difficult to see that a utility function leads to a *preference ordering* over outcomes. For example, if ω and ω' are both possible outcomes in Ω, and $u_i(\omega) \geq u_i(\omega')$, then agent i prefers outcome ω at least as much as outcome ω'. We can introduce a bit more notation to capture this preference ordering. We write

PREFERENCES

$$\omega \succcurlyeq_i \omega'$$

as an abbreviation for

$$u_i(\omega) \geq u_i(\omega')$$

Similarly, if $u_i(\omega) > u_i(\omega')$, then outcome ω is *strictly preferred* by agent i over ω'. We write

STRICT PREFERENCE

$$\omega \succ_i \omega'$$

as an abbreviation for

$$u_i(\omega) > u_i(\omega').$$

In other words,

$$\omega \succ_i \omega' \text{ if and only if } u_i(\omega) > u_i(\omega') \text{ and not } u_i(\omega) = u_i(\omega').$$

We can see that the relation $\succcurlyeq_i$ really is an ordering, in that it has the following properties.

Reflexivity For all $\omega \in \Omega$, we have that $\omega \succcurlyeq_i \omega$.

Transitivity If [illegible]

Transitivity If $\omega \succcurlyeq_i \omega'$, and $\omega' \succcurlyeq_i \omega''$, then $\omega \succcurlyeq_i \omega''$.

Comparability For all $\omega \in \Omega$ and $\omega' \in \Omega$ we have that either $\omega \succcurlyeq_i \omega'$ or $\omega' \succcurlyeq_i \omega$.

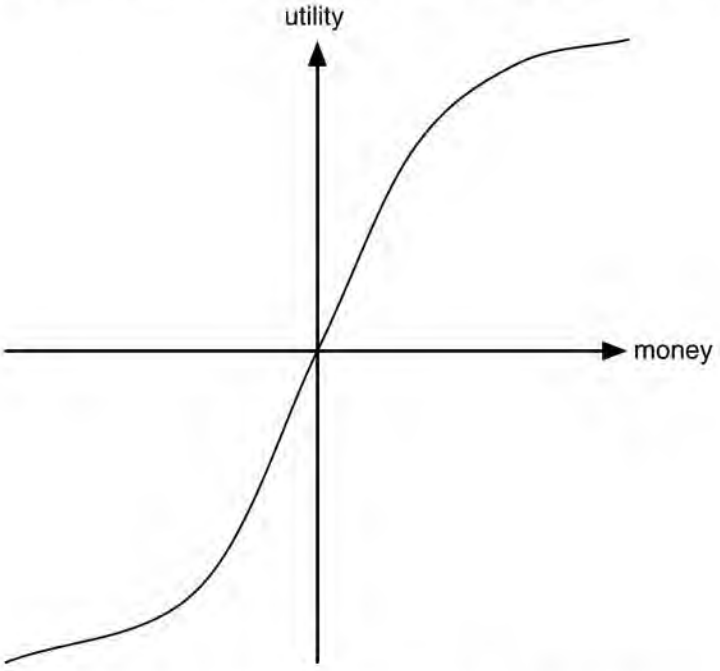
The strict preference relation will satisfy the second and third of these properties, but will clearly not be reflexive.

What is utility?

Undoubtedly the simplest way to think about utilities is as money; the more money, the better. But resist the temptation to think that this is all that utilities are. Utility functions are *just a way of representing an agent's preferences*. They *do not* simply equate to money.

To see what I mean by this, suppose (and this really *is* a supposition) that I have $500 million in the bank, while you are absolutely penniless. A rich benefactor appears, with one million dollars, which he generously wishes to donate to one of us. If the benefactor gives the money to me, what will the increase in the utility of my situation be? Well, I will have more money, so there will clearly be *some* increase in the utility of my situation. But there will not be much: after all, there is not much that you can do with $501 million that you cannot do with $500 million. In contrast, if the benefactor gave the money to you, the increase in your utility would be *enormous*; you would go from having no money at all to being a millionaire. That is a *big* difference.

Figure 11.2: The relationship between money and utility.



This works the other way as well. Suppose I am in *debt* to the tune of $500 million; well, there is frankly not that much difference in utility between owing $500 million and owing $499 million; they are both pretty bad. In contrast, there is a very big difference between owing $1 million and not being in debt at all. A typical graph of the relationship between utility and money is shown in Figure 11.2.

11.2 Setting the Scene

Now that we have our model of agents' preferences, we need to introduce a model of the environment in which these agents will act. The idea is that our two agents will simultaneously choose an action to perform in the environment, and as a result of the actions that they select, an outcome in O will result. The *actual* outcome that will result will depend on the particular

an outcome in Ω will result. The *actual* outcome that will result will depend on the particular *combination* of actions performed. Thus *both* agents can influence the outcome. We will also assume that the agents have no choice about whether to perform an action – they have to simply go ahead and perform one. Further, it is assumed that they cannot see the action performed by the other agent.

To make the analysis a bit easier, we will assume that each agent has just two possible actions that it can perform. We will call these two actions *'C'*, for 'cooperate', and *'D'*, for 'defect'. (The rationale for this terminology will become clear below.) Let $Ac = \{C, D\}$ be the set of these actions. The way the *environment* behaves is then determined by a function

$$\tau : \underbrace{Ac}_{\text{agent } i\text{'s action}} \times \underbrace{Ac}_{\text{agent } j\text{'s action}} \rightarrow \Omega.$$

(This is essentially a state transformer function, as discussed in Chapter 2.) In other words, on the basis of the action (either C or D) selected by agent i, and the action (also either C or D) chosen by agent j an outcome will result.

Here is an example of an environment function:

$$\tau(D,D) = \omega_1, \quad \tau(D,C) = \omega_2, \quad \tau(C,D) = \omega_3, \quad \tau(C,C) = \omega_4. \tag{11.1}$$

This environment maps each combination of actions to a *different* outcome, and is therefore sensitive to the actions of *both* agents. At the other extreme, we can consider an environment that maps each combination of actions to the *same* outcome:

$$\tau(D,D) = \omega_1, \quad \tau(D,C) = \omega_1, \quad \tau(C,D) = \omega_1, \quad \tau(C,C) = \omega_1. \tag{11.2}$$

In this environment, it does not matter what the agents do: the outcome will be the same. Neither agent has any influence in such a scenario. We can also consider an environment that is only sensitive to the actions performed by one of the agents:

$$\tau(D,D) = \omega_1, \quad \tau(D,C) = \omega_2, \quad \tau(C,D) = \omega_1, \quad \tau(C,C) = \omega_2. \tag{11.3}$$

In this environment, it does not matter what agent i does: the outcome depends solely on the action performed by j. If j chooses to defect, then outcome ω_1 will result; if j chooses to cooperate, then outcome ω_2 will result.

The story gets interesting when we put an environment together with the preferences that agents have. To see what I mean by this, suppose that we have the most general case, characterized by (11.1), where both agents are able to exert some influence over the environment. Now let us suppose that the agents have utility functions defined as follows:

$$\left.\begin{array}{llll} u_i(\omega_1) = 1, & u_i(\omega_2) = 1, & u_i(\omega_3) = 4, & u_i(\omega_4) = 4, \\ u_j(\omega_1) = 1, & u_j(\omega_2) = 4, & u_j(\omega_3) = 1, & u_j(\omega_4) = 4. \end{array}\right\} \tag{11.4}$$

Since we know that every different combination of choices by the agents is mapped to a different outcome, we can abuse notation somewhat by writing the following:

$$\left.\begin{array}{llll} u_i(D,D) = 1, & u_i(D,C) = 1, & u_i(C,D) = 4, & u_i(C,C) = 4, \\ u_j(D,D) = 1, & u_j(D,C) = 4, & u_j(C,D) = 1, & u_j(C,C) = 4. \end{array}\right\} \quad (11.5)$$

We can then characterize agent i's preferences over the possible outcomes in the following way:

$$(C, C) \succcurlyeq_i (C, D) \succ_i (D, C) \succcurlyeq_i (D, D).$$

Now, consider the following question.

If you were agent i in this scenario, what would you choose to do – cooperate or defect?

In this case (I hope), the answer is pretty unambiguous. Agent i prefers *all* the outcomes in which it cooperates over *all* the outcomes in which it defects. Agent i's choice is thus clear: it should cooperate. It does not matter, in this case, what agent j chooses to do.

In just the same way, agent j prefers all the outcomes in which *it* cooperates over all the outcomes in which it defects. Notice that, in this scenario, neither agent has to expend any effort worrying about what the other agent will do: the action it should perform does not depend in any way on what the other does.

If both agents in this scenario act rationally, that is, they both choose to perform the action that will lead to their preferred outcomes, then the 'joint' action selected will be (C, C): both agents will cooperate.

Now suppose that, for the same environment, the agents' utility functions are as follows:

$$\left.\begin{array}{llll} u_i(D,D) = 4, & u_i(D,C) = 4, & u_i(C,D) = 1, & u_i(C,C) = 1, \\ u_j(D,D) = 4, & u_j(D,C) = 1, & u_j(C,D) = 4, & u_j(C,C) = 1. \end{array}\right\} \quad (11.6)$$

Agent i's preferences over the possible outcomes are thus as follows:

$$(D, D) \succcurlyeq_i (D, C) \succ_i (C, D) \succcurlyeq_i (C, C).$$

In this scenario, agent i can do no better than to defect. The agent prefers *all* the outcomes in which it defects over *all* the outcomes in which it cooperates. Similarly, agent j can do no better than defect: it also prefers all the outcomes in which it defects over all the outcomes in which it cooperates. Once again, the agents do not need to engage in *strategic* thinking (worrying about what the other agent will do): the best action to perform is entirely independent of the other agent's choice. I emphasize that in most multiagent scenarios the choice an agent should make is not so clear cut; indeed, most are much more difficult.

We can neatly summarize the previous interaction scenario by making use of a standard game-theoretic notation known as a *pay-off matrix:*

PAY-OFF MATRIX

	i defects	i cooperates
j defects	4 4	1 4
j	1	1

This pay-off matrix in fact defines a *game* (to be precise, what is technically called a 'game in strategic form'). The way to read such a pay-off matrix is as follows. Each of the four cells in the matrix corresponds to one of the four possible outcomes. For example, the top-right cell corresponds to the outcome in which i cooperates and j defects; the bottom-left cell corresponds to the outcome in which i defects and j cooperates. The pay-offs received by the two agents are written in the cell. The value in the top right of each cell is the payoff received by player i (the *column player*), while the value in the bottom left of each cell is the pay-off received by agent j (the *row player*). As pay-off matrices are standard in the literature, and are a much more succinct notation than the alternatives, we will use them as standard in the remainder of this chapter.

John Forbes Nash, Jr.

John Forbes Nash, Jr. seems a textbook example of the cliché that genius goes hand-in-hand with mental turmoil. Born in 1928, Nash joined Princeton University as a postgraduate student in 1948, and within two years gained his PhD: his thesis was a mere 28 pages long. As somebody who has supervised a number of PhD students over the years, I find it very hard indeed to imagine supervising somebody so far away from our standard benchmarks of intellect. (It is also quite hard to imagine supervising a PhD student with such self confidence that he dropped in on Albert Einstein to give him advice on how to do physics, as the young Nash apparently did [Binmore, 2007, p. 29]!) Anyway, in the space of three years, four published papers, and one 28-page PhD thesis, Nash opened up a completely new avenue for understanding rational action in games, most notably through the concept that we now call 'Nash equilibrium'. Until Nash formulated this idea, game theory had focused on two-person zero-sum games, and the associated minimax theorem, proved by celebrated Hungarian polymath John von Neumann in the year Nash was born. The minimax theorem gives an elegant and intuitive solution to certain games, but the class of games to which it may be applied is rather limited. There simply was no framework through which to understand many other types of games. Nash equilibrium provided such a framework, and expanded enormously the applicability of game-theoretic analysis. Unfortunately, Nash's mental state deteriorated after his PhD work, until, less than a decade after formulating some of the most important ideas in 20th century economics, he was admitted to hospital suffering from schizophrenia. Mental illness is often misunderstood and badly treated today, half a century later. But in the 1950s, understanding and treatment were considerably cruder. Nash struggled with mental illness for decades, effectively placing him outside academia and largely unable to work, while the intellectual seeds he had planted with his PhD work grew and grew. They became perhaps *the* central building blocks of game theory, and have applicability in areas far beyond those which even the genius Nash could have anticipated. Fortunately, by 1994 Nash was sufficiently recovered to be able to accept the Nobel Prize for Economics, jointly with John Harsanyi and Reinhard Selten. His brilliant work and troubled life were brought to the attention of a non-academic audience by Sylvia Nasar in her 1998 biography *A Beautiful Mind*, made into an Oscar-winning film in 2001.

11.3 Solution Concepts and Solution Properties

Given a particular multiagent encounter involving two agents i and j, the basic question that both agents want answered is: *what should I do*? We have already seen some multiagent

encounters, and informally argued what the best possible answer to this question should be. In this section, we will define some of the concepts that are used in answering this question.

11.3.1 Dominant strategies

The first concept we will introduce is that of *dominance*. We will say that a strategy s_i is *dominant* for player i if, no matter what strategy s_j agent j chooses, i will do at least as well playing s_i as it would doing anything else. We can also explain this idea in the concept of *best response*. Suppose agent j plays strategy s_j; then agent i's best response to s_j is simply the strategy that gives i the highest pay-off when played against s_j. A strategy s_i for agent i is dominant if it is the best response to *all* of agent j's strategies. If we consider the scenario in Equations (11.5), it should be clear that, for both agents, cooperation is the dominant strategy. The presence of a dominant strategy makes the decision about what to do extremely easy: the agent guarantees its best outcome by performing the dominant strategy.

DOMINANT STRATEGIES

BEST RESPONSE

11.3.2 Nash equilibria

The next notion we shall discuss is one of the most important concepts in the game-theory literature, and in turn is one of the most important concepts in analysing multiagent systems. The notion is that of *equilibrium*, and, more specifically, *Nash equilibrium*. The intuition behind equilibrium is perhaps best explained by example. Every time you drive a car, you need to decide which side of the road to drive on. The choice is not a very hard one: if you are in the UK, for example, you will probably choose to drive on the left; if you are in the USA or continental Europe, you will drive on the right. The reason the choice is not hard is that it forms part of a Nash equilibrium: assuming everyone else is driving on the left, you can do no better than drive on the left also; and from the point of view of everyone else, assuming you are driving on the left, then they can do no better than drive on the left also.

NASH EQUILIBRIUM

In general, we will say that two strategies s_1 and s_2 are in Nash equilibrium if:

1. under the assumption that agent i plays s_1, agent j can do no better than play s_2, and
2. under the assumption that agent j plays s_2, agent i can do no better than play s_1.

To put it another way, strategies s_i and s_j for agents i and j form a Nash equilibrium *if they are the best response to each other*.

The *mutual* form of an equilibrium is important because it 'locks the agents in' to a pair of strategies. *Neither agent has any incentive to deviate from a Nash equilibrium*. To see why, suppose s_1, s_2 are a pair of strategies in Nash equilibrium for agents i and j, respectively, and that agent i chooses to play some other strategy, s_3 say. Then by definition, i will do no better (and may possibly do worse) than it would have done by playing s_1.

PURE STRATEGY NASH EQUILIBRIUM

Technically, this type of Nash equilibrium is known as *pure strategy Nash equilibrium*. From a computational point of view, checking for the existence of a pure strategy Nash equilibrium is easy to implement. We simply consider each possible combination of strategies in turn, and, for each combination, check whether this combination forms a best response for every agent. Now, if there are n agents, each with m possible strategies, then the number of possible combinations of actions (and hence possible outcomes) will be m^n, and so this naive approach will be exponential in the number of agents. But, if the number of agents is fixed or small, then this will be acceptable.

The presence of a pure strategy Nash equilibrium might appear to be the definitive answer to the question of what to do in any given scenario. Unfortunately, there are two important problems with pure strategy Nash equilibria which limit their use:

1. Not every interaction scenario has a pure strategy Nash equilibrium.
2. Some interaction scenarios have more than one pure strategy Nash equilibrium.

Before proceeding, let us see an example of a game that has no pure strategy Nash equilibrium. The game is called *matching pennies*:

> Players i and j simultaneously choose the face of a coin, either 'heads' or 'tails'. If they show the same face, then i wins, while if they show different faces, then j wins.

Expressed as a pay-off matrix, matching pennies looks like this:

	i heads		i tails	
j heads		−1		1
	1		−1	
j tails		1		−1
	−1		1	

You can see that there is no pure strategy Nash equilibrium by considering each cell in turn. For example, if the outcome was (*heads, heads*), then player j would regret their decision, since they would have preferred to have chosen *tails*; on the other hand, if the outcome was (*heads, tails*), then player i would regret their decision, and would have preferred to have chosen *tails*; and so on. No matter which outcome we consider, one of the agents would have preferred to have made another choice, under the assumption that the other player did not change their choice.

Mixed strategies and Nash's theorem

To overcome the first of these problems, and to see the real power of Nash equilibrium, we have to modify our idea of strategy a little bit. We have been thinking of strategies simply as actions, or subroutines that an agent can execute. When we write a subroutine, we do not usually want any *uncertainty* or *randomness* in how that subroutine executes: we want exactly the same behaviour for our program every time we execute it. But sometimes, *it can be useful to introduce randomness or uncertainty into our actions*. To see this, let's consider the playground game *rock–paper–scissors*. This game is played by two players, as follows. The players must simultaneously declare one of three possible choices: rock, scissors, or paper. (They usually do this by showing a fist for rock, an open palm for paper, or a V-shape

with the fingers for scissors.) To determine the winner, the following rule is used:

Paper covers rock; scissors cut paper; rock blunts scissors.

In other words, if you declare paper and I declare rock, then you win; if you declare paper and I declare scissors, then I win; if I declare scissors and you declare rock, then you win, and so on. If we make the same choice, then we draw. Here is the pay-off matrix for this game.

	i plays rock	*i* plays paper	*i* plays scissors
j plays rock	0 1	0 1	−1 −1
j plays paper	1 0	−1 −1	0 1
j plays scissors	−1 −1	1 0	1 0

So what should you do in such a game? Whatever choice you make, you can end up with the worst possible outcome: a utility of −1. There is no dominant strategy. Moreover, there is no 'stable point' in the game: whichever cell in the pay-off matrix you choose, at least one of you would have preferred to make another choice, assuming the other player did not change their choice. For example, if you played rock while I played scissors, then I would regret my choice, and would have preferred to play paper. There is no pure strategy Nash equilibrium.

You might flippantly answer that you may as well choose a strategy at random in a game like this. Well, it turns out that this is in fact a rather good idea! A *mixed strategy* allows you to choose between possible choices *by introducing randomness* into the selection. In rock–paper–scissors, the crucial mixed strategy is as follows: *choose between rock, paper, and scissors at random, with each choice having equal probability of being selected*. It turns out that this strategy is in Nash equilibrium with itself! That is, if this is how I play the game, then you can do no better than play the game using this strategy as well, and vice versa. Note that the clause '… with each choice having equal probability of being selected' is very important. If I play this game with you frequently, and see that you are favouring one choice – scissors, for example – over another, then I can exploit this fact, by favouring rock in my strategy.

MIXED STRATEGY

In general, if a player has k possible choices, $s_1, s_2, \ldots, s_k$, then a mixed strategy over these choices takes the form:

- play s_1 with probability p_1
- play s_2 with probability p_2
- …
- play s_k with probability p_k.

Of course, since these are probabilities, then we will obviously require that $p_1 + p_2 + \ldots + p_k = 1$. Expressed somewhat more formally, a mixed strategy over $s_1, s_2, \ldots, s_k$ is a probability distribution over $s_1, s_2, \ldots, s_k$.

distribution over $s_1, s_2, \ldots, s_k$.

Now, if we consider mixed strategies, then the landscape of Nash equilibrium changes dramatically: the following result was proved by John Forbes Nash, Jr.:

NASH'S THEOREM

Every game in which every player has a finite set of possible strategies has a Nash equilibrium in mixed strategies.

The Nobel prize committee think this is a very important result: after all, they gave Nash a Nobel Prize for it (see sidebar, *John Forbes Nash, Jr.*).

What about computational aspects of Nash equilibria in mixed strategies? The problem turns out to be rather interesting from a computational point of view. In the theory of computational complexity, the most common formulation of a problem is as a *decision problem:* a problem to which the answers can be either 'yes' or 'no'. For example, in the decision variant of the well-known 'travelling salesman' problem, the question asked is 'does there exist a tour including all cities with total cost less than or equal to k?' In the SAT problem, the question asked is 'does this formula have a satisfying assignment?' In both cases, the answer is either 'yes' or 'no', and in general, both answers are possible. Some weighted graphs have a tour including all cities with total cost less than or equal to k; others do not. Some formulae have a satisfying assignment; others do not. However, asking 'does this game have a Nash equilibrium?' doesn't make a lot of sense: Nash's theorem tells us the answer is always 'yes'! Problems like this are not too well understood in the field of computational complexity; they are called *total search problems*, and the usual yes/no-oriented complexity classes (P, NP, etc.) simply aren't appropriate for such problems.

TOTAL SEARCH PROBLEM

In fact, the complexity of computing Nash equlibria in mixed strategies became, in the early part of the 21st century, a rather celebrated problem:

> [T]he complexity of finding a Nash equilibrium is the most important concrete open question on the boundary of P today.
>
> [Papadimitriou, 2001]

While a number of related results about computing Nash equilibria were proved (e.g. complexity of finding Nash equilibria with certain properties, and complexity of Nash equilibria on various classes of games [Conitzer and Sandholm, 2003, 2008]), the central problem resisted resolution until a series of papers appeared between 2005 and 2006. The first breakthrough result was [Daskalakis et al., 2006], which proved that computing Nash equilibria in four-player games was *PPAD-complete*. 'PPAD' is a rather obscure complexity class (at least in comparison with well-known classes like P and NP), specifically intended to characterize total search problems. This result was then eventually strengthened to two-player games in [Chen and Deng, 2006].

11.3.3 Pareto efficiency

Pareto efficiency (also called Pareto optimality) is not really a solution concept so much as a property of solutions, but it is useful to introduce it at this point, as a criterion that desirable

solutions should satisfy. We will say that an outcome is Pareto efficient if there is no other outcome that improves one player's utility without making somebody else worse off. Conversely, an outcome is said to be Pareto inefficient if there is another outcome that makes at least one player better off without making anybody else worse off. A Pareto inefficient outcome is inefficient in the sense that some utility is 'wasted': we could have had another outcome that made somebody better off, and nobody would have objected to changing to this outcome! Notice that while Pareto efficiency is an important property of outcomes, it is perhaps not very useful as a way of selecting outcomes. For example, suppose a brother and sister are playing the well-known childhood 'game' of dividing a chocolate cake among themselves. The outcome in which the sister gets the whole cake is Pareto efficient: any other outcome would be worse, from the point of view of the sister. But it is hard to see this as a 'reasonable' outcome. (As an aside, *any* way of dividing a cake between the brother and sister in which the whole cake is given out will be Pareto optimal, because any other outcome would give more to one but less to the other: so either the brother or the sister would always object.)

A Game for Monkeys?

Rock–paper–scissors is such a simple game that children can play it. But can *monkeys?* This unlikely sounding question was in fact the subject of a 2005 article published in the journal *Cognitive Brain Research* [Lee et al., 2005]. The authors conducted a series of experiments to see if monkeys could learn to play rock–paper–scissors, and if so, whether the way they played could be explained by game theory or by some other model. In fact, the authors had previously published the results of experiments with monkeys playing the game of matching pennies: they had observed that monkeys learned to come close to the Nash equilibrium outcome (choose heads or tails with equal probability), but that they never lost a bias completely. In the rock–paper–scissors experiment, two male rhesus monkeys played the game against a computer. Three increasingly sophisticated algorithms were used against the monkeys. The first corresponded to the Nash equilibrium (choose at random, with equal probability); the second tried to compute the probability that a monkey would choose an outcome based on their previous choices, and use this to choose an outcome to beat the monkey's choice. The third algorithm was a more sophisticated variation of this. The notion of pay-off was implemented by rewarding the monkeys with drinks of juice. The monkeys were tested over a 31-day period, with each monkey playing up to 2189 trials per day. These experiments may seem a little comical – the *Annals of Improbable Research* certainly thought so, for they featured the experiments on their website. (Some would also regard the experiments as cruel, although a discussion of this issue would take us off at a tangent.) But there is a serious purpose to them. There is a fierce debate among researchers about the value of solution concepts such as Nash equilibrium, as experimental evidence suggests that people often don't use such outcomes in practice. The monkey experiments were intended to study what kinds of model successfully predict how people actually play. The authors found that 'each animal displayed an idiosyncratic pattern substantially deviating from the Nash equilibrium'. They concluded that the behaviour of monkeys during a competitive game can be described better by a model based on *reinforcement learning*.

11.3.4 Maximizing social welfare

As with Pareto efficiency, social welfare is an important property of outcomes, but is not generally a way of directly selecting outcomes. The idea is very simple: we measure how much utility is created by an outcome in total. A bit more formally, let $sw(\omega)$ denote the sum of the utilities of each agent for outcome ω:

$$sw(\omega) = \sum_{i \in Ag} u_i(\omega).$$

Obviously, the outcome that maximizes social welfare is the one that maximizes this value.

From an individual agent's point of view, the problem with maximizing social welfare is that it does not look at the pay-offs of individual agents, only the total wealth created. For example, suppose you have a game with 100 players, in which one outcome gives \$101 million to one player and nothing to the rest, while every other outcome gives \$1 million to each. The first outcome – giving all the wealth to just one player – maximizes social welfare, although the other 99 players might not be very happy with such an outcome.

Maximizing social welfare becomes relevant if all the agents within a system have the same 'owner'. In this case, it is not important to worry about how the utility of an outcome is divided among the players. Instead, all we need to worry about is the total overall utility, that is, how the system functions as a whole. Social welfare provides a measure of this.

11.4 Competitive and Zero-Sum Interactions

Suppose we have some scenario in which an outcome $\omega \in \Omega$ is preferred by agent i over an outcome ω' if, and only if, ω' is preferred over ω by agent j. Formally,

$$\omega \succ_i \omega' \text{ if and only if } \omega' \succ_j \omega.$$

The preferences of the players are thus diametrically opposed to one another: one agent can only improve its lot (i.e. get a more preferred outcome) at the expense of the other. An interaction scenario that satisfies this property is said to be *strictly competitive*, for hopefully obvious reasons.

STRICTLY COMPETITIVE

ZERO SUM

Zero-sum encounters are those in which, for any particular outcome, the utilities of the two agents sum to zero. Formally, a scenario is said to be zero sum if the following condition is satisfied:

$$u_i(\omega) + u_j(\omega) = 0 \quad \text{for all } \omega \in \Omega.$$

It should be easy to see that any zero-sum scenario is strictly competitive. Zero-sum encounters are important because they are the most 'vicious' type of encounter conceivable, allowing for no possibility of cooperative behaviour: the best outcome for you is the worst outcome for your opponent. If you allow your opponent to get positive utility, then you get *negative* utility.

Games such as chess and checkers are the most obvious examples of strictly competitive interactions. Indeed, any game in which the possible outcomes are win or lose will be strictly competitive. However, it is hard to think of real-world examples of zero-sum encounters. War is sometimes cited as a zero-sum interaction between nations, but even in the most extreme

wars, there will usually be at least *some* common interest between the participants (e.g. in ensuring that the planet survives). Perhaps games such as rock–paper–scissors (which are after all an artificial and highly stylized form of interaction) are the only real-world examples of zero-sum encounters.

For these reasons, some social scientists are sceptical about whether zero-sum games exist in real-world scenarios [Zagare, 1984, p. 22]. Interestingly, however, people interacting in many scenarios have a tendency to treat them *as if they were zero sum*. This can sometimes have undesirable consequences.

Let us now apply the ideas we have been developing above to some actual multiagent scenarios. First, let us consider what is perhaps the best-known multiagent scenario: the *prisoner's dilemma*.

11.5 The Prisoner's Dilemma

PRISONER'S DILEMMA

Consider the following scenario.

> Two men are collectively charged with a crime and held in separate cells. They have no way of communicating with each other or making any kind of agreement. The two men are told that:
>
> 1. if one of them confesses to the crime and the other does not, the confessor will be freed, and the other will be jailed for three years
> 2. if both confess to the crime, then each will be jailed for two years.
>
> Both prisoners know that if neither confesses, then they will each be jailed for one year.

We refer to confessing as defection, and not confessing as cooperating. Before reading any further, stop and think about this scenario: if you were one of the prisoners, what would you do? (Write down your answer somewhere, together with your reasoning; after you have read the discussion below, return and see how you fared.)

There are four possible outcomes to the prisoner's dilemma, depending on whether the agents cooperate or defect, and so the environment is of type (11.1). Abstracting from the scenario above, we can write down the utility functions for each agent in the following pay-off matrix:

	i defects	*i* cooperates
j defects	2 2	5 0
j cooperates	0 5	3 3

Note that the numbers in the pay-off matrix do not refer to years in prison. They capture how good an outcome is for the agents – the shorter the jail term, the better.

In other words, the utilities are

$$u_i(D, D) = 2,\ u_i(D, C) = 5,\ u_i(C, D) = 0,\ u_i(C, C) = 3,$$

$$u_j(D, D) = 2,\ u_j(D, C) = 0,\ u_j(C, D) = 5,\ u_j(C, C) = 3,$$

and the preferences are

$$(D, C) \succ_i (C, C) \succ_i (D, D) \succ_i (C, D), \text{ and}$$

$$(C, D) \succ_j (C, C) \succ_j (D, D) \succ_j (D, C).$$

What should a prisoner do? The 'standard' approach to this problem is to put yourself in the place of a prisoner, i say, and reason as follows.

- Suppose the other player cooperates. Then my best response is to defect.
- Suppose the other player defects. Then my best response is to defect.

In other words, defection for i is the best response to all possible strategies of the player j: defection is thus a dominant strategy for i. The scenario is symmetric – both agents reason the same way, and so *both agents will defect*, resulting in an outcome that gives them each a pay-off of 2. Turning to Nash equilibria, we can see that there is a single Nash equilibrium of (D, D): under the assumption that i will play D, j can do no better than play D, and under the assumption that j will play D, i can also do no better than play D.

So, it seems that we are inevitably going to end up with the (D, D) outcome. But *intuition says this is not the best the players can do*. Surely if they both *cooperated*, then they would do better – they would receive a pay-off of 3! But if you assume that the other agent will cooperate, then the rational thing to do is to defect. The conclusion seems inescapable: the rational thing to do in the prisoner's dilemma is defect, even though this 'wastes' some utility. Applying the other principles we discussed above, you should be able to see that (D, D) is the only outcome that is *not* Pareto efficient, while the outcome that maximizes social welfare is (C, C).

The fact that utility seems to be wasted here, and that the agents could both do better by cooperating, even though the rational thing to do is to defect, is why this is referred to as a dilemma.

The prisoner's dilemma may seem an abstract problem, but it turns out to be very common indeed. In the real world, the prisoner's dilemma appears in situations ranging from nuclear weapons treaty compliance to negotiating with one's children. Consider the problem of nuclear weapons treaty compliance. Two countries i and j have signed a treaty to dispose of their nuclear weapons. Each country can then either cooperate (i.e. get rid of their weapons), or defect (i.e. keep their weapons). But if you get rid of your weapons, you run the risk that the other side keeps theirs, making them very well off, while you suffer what is called the 'sucker's pay-off'. In contrast, if you keep yours, then the possible outcomes are that you will have nuclear weapons while the other country does not (a very good outcome for you), or else at worst that you both retain your weapons. This may not be the best possible outcome, but is certainly better than you giving up your weapons while your opponent keeps theirs, which is what you risk if you give up your weapons.

The prisoner's dilemma also seems to be the game that characterizes the *tragedy of the commons* [Hardin, 1968]. The tragedy of the commons is concerned with the use of a shared, depletable resource by a society of self-interested individuals. A standard example used to illustrate the tragedy of the commons is that of an area of common land, which can be used for grazing livestock. If all the land-users make modest use of the land, then the land remains in good condition for future use. However, from the point of view of an individual land-user, the best outcome is obtained if they make intensive use of the land while everyone else makes

modest use. In this case, the user receives high value from the land and it remains in reasonably good condition for the future. Unfortunately, if everybody reasons in this way, and everybody makes intensive use of the land, then the land is overgrazed, and becomes unusable by anybody. Clearly this is a poor outcome for everybody, worse than if everybody used the land modestly. This type of scenario seems to be very common in the real world, and seems to have something to say about scenarios ranging from overfishing in the seas through to exploitation of bandwidth capacity on the Internet [Diamond, 2005].

TRAGEDY OF THE COMMONS

Many people find the conclusion of the analysis we presented above – that the rational thing to do in the prisoner's dilemma is defect – deeply upsetting. The result *seems* to imply that cooperation can only arise as a result of *irrational* behaviour, and that cooperative behaviour can be exploited by those who behave rationally. The apparent conclusion is that nature really is 'red in tooth and claw'. Particularly for those who are inclined to a liberal view of the world, this is unsettling and perhaps even distasteful. As civilized beings, we tend to pride ourselves on somehow 'rising above' the other animals in the world, and believe that we are capable of nobler behaviour: to argue in favour of such an analysis is therefore somehow immoral, and even demeaning to the entire human race. Binmore argues that the discomfort we have with the analysis of the prisoner's dilemma is misplaced:

> A whole generation of scholars swallowed the line that the prisoner's dilemma embodies the essence of the problem of human cooperation. They therefore set themselves the hopeless task of giving reasons why [this analysis] is mistaken…. Rational players don't cooperate in the prisoner's dilemma because the conditions necessary for rational cooperation are absent. [Binmore, 2007, pp. 18–19]

Here are some of the arguments put forward in an attempt to 'recover rational cooperation' in the prisoner's dilemma [Binmore, 1992, pp. 355–382].

We are not all Machiavelli!

The first approach is to argue that we are not all such 'hard-boiled' individuals as the prisoner's dilemma (and more generally, this kind of game-theoretic analysis) implies. We are *not* seeking to constantly maximize our own welfare, possibly at the expense of others. Proponents of this kind of argument typically point to real-world examples of *altruism* and spontaneous, mutually beneficial cooperative behaviour in order to justify their claim.

There is some strength to this argument: we do not (or at least, most of us do not) constantly deliberate about how to maximize our welfare without any consideration for the welfare of our peers. Similarly, in many scenarios, we would be happy to trust our peers to recognize the value of a cooperative outcome without even mentioning it to them, being no more than mildly annoyed if we get the 'sucker's pay-off'.

There are several counter responses to this. First, it is pointed out that many real-world examples of spontaneous cooperative behaviour are not really the prisoner's dilemma. Frequently, there is some built-in mechanism that makes it in the interests of participants to cooperate. For example, consider the problem of giving up your seat on the bus. We will frequently give up our seat on the bus to an older person, a mother with children, etc., apparently at some discomfort (i.e. loss of utility) to ourselves. But it could be argued that in such scenarios, society has ways of punishing non-cooperative behaviour: suffering the hard

such scenarios, society has ways of punishing non-cooperative behaviour: suffering the hard and unforgiving stares of fellow passengers when we do not give up our seat, or worse, being publicly accused of being uncouth!

Second, it is argued that many 'counter-examples' of cooperative behaviour arising do not stand up to inspection. For example, consider a public transport system that relies on everyone cooperating and honestly paying their fare every time they travel, even though whether they have paid is not verified. The fact that such a system works would appear to be evidence that relying on spontaneous cooperation can work. But the fact that such a system works does not mean that it is not exploited. It will be, and if there is no means of checking whether or not someone has paid their fare and punishing non-compliance, then all other things being equal, those individuals that do exploit the system (defect) will be better off than those that pay honestly (cooperate). Unpalatable, perhaps, but true nevertheless.

The other prisoner is my twin!

A second line of attack is to argue that the two prisoners will 'think alike', and recognize that cooperation is the best outcome. For example, suppose the two prisoners are twins, unseparated since birth; then, it is argued, if their thought processes are sufficiently aligned, they will both recognize the benefits of cooperation, and behave accordingly. The answer to this is that it implies there are not actually two prisoners playing the game. If I can make my twin select a course of action simply by 'thinking it', then we are not playing the prisoner's dilemma at all.

This 'fallacy of the twins' argument often takes the form 'what if everyone were to behave like that' [Binmore, 1992, p. 311]. The answer (as Yossarian pointed out in Joseph Heller's *Catch 22*) is that if everyone else behaved like that, you would be a damn fool to behave any other way.

People are not rational!

Some people would argue that we might indeed be happy to risk cooperation as opposed to defection when faced with situations where the sucker's pay-off really does not matter very much. For example, paying a bus fare that amounts to a few pennies does not really hurt us much, even if everybody else is defecting and hence exploiting the system. In such cases, people are often not strictly rational. But, when we are faced with situations where the sucker's pay-off really *hurts* us – life or death situations and the like – we will tend to choose the 'rational' course of action.

11.5.1 The shadow of the future

There are in fact quite natural variants of the prisoner's dilemma in which cooperation *can* be the rational thing to do. One idea is to *play the game more than once*. In the *iterated prisoner's dilemma*, the 'game' of the prisoner's dilemma is played a number of times. Each play is referred to as a 'round'. Critically, it is assumed that each agent can see what the opponent did on the previous round: player i can see whether j defected or not, and j can see whether i defected or not.

ITERATED PRISONER'S DILEMMA

Now, for the sake of argument, assume that the agents will continue to play the game *forever*: every round will be followed by another round. Now, under these assumptions, what is the

rational thing to do? If you know that you will be meeting the same opponent in future rounds, the incentive to defect appears to be considerably diminished, for two reasons.

- If you defect now, your opponent can *punish* you by also defecting. Punishment is not possible in the one-shot prisoner's dilemma.
- If you 'test the water' by cooperating initially, and receive the sucker's pay-off on the first round, then because you are playing the game indefinitely, this loss of utility (one util) can be 'amortized' over the future rounds. When taken into the context of an infinite (or at least very long) run, then the loss of a single unit of utility will represent a small percentage of the overall utility gained.

So, if you play the prisoner's dilemma game indefinitely, then cooperation is a rational outcome [Binmore, 1992, p. 358]. The 'shadow of the future' encourages us to cooperate in the infinitely repeated prisoner's dilemma game.

This seems to be very good news indeed, as truly one-shot games are comparatively scarce in real life. When we interact with someone, then there is often a good chance that we will interact with them in the future, and rational cooperation begins to look possible. However, there is a catch.

Suppose you agree to play the iterated prisoner's dilemma a *fixed* number of times (say 100). You need to decide (presumably in advance) what your strategy for playing the game will be. Consider the last round (i.e. the 100th game). Now, on this round, you know (as does your opponent) that you will not be interacting again. In other words, the last round is in effect a one-shot prisoner's dilemma game. As we know from the analysis above, the rational thing to do in a one-shot prisoner's dilemma game is defect. Your opponent, as a rational agent, will presumably reason likewise, and will also defect. On the 100th round, therefore, you will both defect. But this means that the last 'real' round is 99. But similar reasoning leads us to the conclusion that this round will also be treated in effect like a one-shot prisoner's dilemma, and so on. Continuing this *backward induction* argument leads inevitably to the conclusion that, in the iterated prisoner's dilemma with a fixed, predetermined, commonly known number of rounds, defection is the dominant strategy, as in the one-shot version [Binmore, 1992, p. 354].

BACKWARD INDUCTION

Whereas it seemed to be very good news that rational cooperation is possible in the iterated prisoner's dilemma with an infinite number of rounds, it seems to be very bad news that this possibility appears to evaporate if we restrict ourselves to repeating the game a predetermined, fixed number of times. Returning to the real world, we know that in reality, we will only interact with our opponents a finite number of times (after all, one day the world will end). We appear to be back where we started.

The story is actually better than it might at first appear, for several reasons. The first is that *actually* playing the game an infinite number of times is not necessary. As long as the 'shadow of the future' looms sufficiently large, then it can encourage cooperation. So, rational cooperation can become possible if both players know, with sufficient probability, that they will meet and play the game again in the future.

The second reason is that, even though a cooperative agent can suffer when playing against a defecting opponent, it can do well overall provided it gets sufficient opportunity to interact

defecting opponent, it can do well overall provided it gets sufficient opportunity to interact with other cooperative agents. To understand how this idea works, we will now turn to one of the best-known pieces of multiagent systems research: Axelrod's prisoner's dilemma tournament.

Axelrod's tournament

Robert Axelrod was (indeed, is) a political scientist interested in how cooperation can arise in societies of self-interested agents. In 1980, he organized a public tournament in which political scientists, psychologists, economists, and game theoreticians were invited to submit a computer program to play the iterated prisoner's dilemma. Each computer program had available to it the previous choices made by its opponent, and simply selected either *C* or *D* on the basis of these. Each computer program was played against each other for five games, each game consisting of 200 rounds. The 'winner' of the tournament was the program that did best *overall*, i.e. best when considered against the whole range of programs. The computer programs submitted ranged from 152 lines of program code to just five lines; a number of 'control' strategies were also included. Here are some examples of the kinds of strategy that were submitted.

AXELROD'S TOURNAMENT

RANDOM This strategy is a control: it ignores what its opponent has done on previous rounds, and selects either *C* or *D* at random, with equal probability of either outcome.

ALL-D This is the 'hawk' strategy, which encodes what a game-theoretic analysis tells us is the 'rational' strategy in the one-shot prisoner's dilemma: always defect, no matter what your opponent has done.

TIT-FOR-TAT This strategy is as follows:

TIT-FOR-TAT

1. on the first round, cooperate
2. on round $t > 1$, do what your opponent did on round $t - 1$.

TIT-FOR-TAT was actually the simplest strategy entered, requiring only five lines of Fortran code.

TESTER This strategy was intended to exploit computer programs that did not punish defection: as its name suggests, on the first round it tested its opponent by defecting. If the opponent ever retaliated with defection, then it subsequently played TIT-FOR-TAT. If the opponent did not defect, then it played a repeated sequence of cooperating for two rounds, and then defecting.

JOSS Like TESTER, the JOSS strategy was intended to exploit 'weak' opponents. It is essentially TIT-FOR-TAT, but 10% of the time, instead of cooperating, it will defect.

Before proceeding, consider the following two questions.

1. On the basis of what you know so far, and, in particular, what you know of the game-theoretic results relating to the finitely iterated prisoner's dilemma, which

strategy do you think would do best overall?

2. If you were entering the competition, which strategy would you enter?

After the tournament was played, the result was that the overall winner was TIT-FOR-TAT. At first sight, this result seems extraordinary. It appears to be empirical proof that the game-theoretic analysis of the iterated prisoner's dilemma is wrong: cooperation *is* the rational thing to do, after all! But the result, while significant, is more subtle (and possibly less encouraging) than this. TIT-FOR-TAT won because the overall score was computed by taking into account *all* the strategies that it played against. The result when TIT-FOR-TAT was played against ALL-D was exactly as might be expected: ALL-D came out on top. Many people have misinterpreted these results as meaning that TIT-FORTAT is the optimal strategy in the iterated prisoner's dilemma. *You should be careful not to interpret Axelrod's results in this way*. TIT-FOR-TAT was able to succeed because it had the opportunity to play against other programs that were also inclined to cooperate. Provided the environment in which TIT-FOR-TAT plays contains sufficient opportunity to interact with other 'like-minded' strategies, TIT-FOR-TAT can prosper. The TIT-FOR-TAT strategy will not prosper if it is forced to interact with strategies that tend to defect.

Axelrod attempted to characterize the reasons for the success of TIT-FOR-TAT, and came up with the following four rules for success in the iterated prisoner's dilemma.

(1) Do not be envious In the prisoner's dilemma, it is not necessary for you to 'beat' your opponent in order for you to do well.

(2) Do not be the first to defect Axelrod refers to a program as 'nice' if it starts by cooperating. He found that whether or not a rule was nice was the single best predictor of success in his tournaments. There is clearly a risk in starting with cooperation. But the loss of utility associated with receiving the sucker's pay-off on the first round will be comparatively small compared with the possible benefits of mutual cooperation with another nice strategy.

(3) Reciprocate cooperation and defection As Axelrod puts it, 'TIT-FOR-TAT represents a balance between punishing and being forgiving' [Axelrod, 1984, p. 119]: the combination of punishing defection and rewarding cooperation seems to encourage cooperation. Although TIT-FOR-TAT can be exploited on the first round, it retaliates relentlessly for such non-cooperative behaviour. Moreover, TIT-FOR-TAT punishes with *exactly* the same degree of violence that it was the recipient of: in other words, it never 'overreacts' to defection. In addition, because TIT-FOR-TAT is *forgiving* (it rewards cooperation), it is possible for cooperation to become established even following a poor start.

(4) Do not be too clever As noted above, TIT-FOR-TAT was the simplest program entered into Axelrod's competition. Either surprisingly or not, depending on your point of view, it fared significantly better than other programs that attempted to make use of comparatively advanced programming techniques in order to decide what to do. Axelrod suggests three reasons for this:

1. The most complex entries attempted to develop a model of the behaviour of the other agent while ignoring the fact that this agent was in turn watching the original agent – they lacked a model of the reciprocal learning that actually takes

place.

2. Most complex entries over-generalized when seeing their opponent defect, and did not allow for the fact that cooperation was still possible in the future – they were not *forgiving*.
3. Many complex entries exhibited behaviour that was too complex to be understood – to their opponent, they may as well have been acting randomly.

11.5.2 Program equilibria

One of the frustrations with the prisoner's dilemma is that the apparent 'solution' of (*C, C*) seems to be so very close. You would both *prefer* the outcome (*C, C*) to (*D, D*), but you cannot cooperate because you have to assume that the other player will defect. How can you protect yourself against the sucker's pay-off? Intuitively, what you want to do is not play a strategy that says 'I'll cooperate', but rather one that says *'I'll cooperate if you will'*. The difficulty is making this idea concrete. The concept of *program equilibria* does this, and does it rather elegantly [Tennenholtz, 2004].

PROGRAM EQUILIBRIA

The idea behind program equilibria is as follows. Each player in the game is permitted a strategy that is not simply of the form *C* or *D* (or whatever strategies/actions are allowed in the game), but is rather a *program*, which is allowed to contain many of the usual constructs that we find in conventional programming languages. Crucially, these strategy programs are permitted to *compare themselves to each other*. Comparison in this sense simply means that the *program text* is compared, just as if we were doing a string comparison in a regular programming language; we compare the source code of the programs to see if they are identical. Within the program, the variable `P1` will refer to the text of the program submitted by player 1, while `P2` refers to the text of the program submitted by player 2, and `==` denotes equality of program texts. The instruction `DO(`α`)` means 'perform action α', and `STOP`, unsurprisingly, means 'stop'. Each player simultaneously submits their program to a *mediator*, and these programs are then jointly executed by the mediator, with the output being a collection of actions, one for each program (see Figure 11.3).

MEDIATOR

Now, suppose player 1 submits the following program strategy:

Figure 11.3: Program equilibria. (a) If both players submit programs that cooperate only if the other player submits the same program, then cooperation will result. (b) These programs are not susceptible to the sucker's pay-off: if one simply defects, then the result will be that the other also defects.

Player 1 (P1):

```
IF P1 == P2
THEN
  DO(C);
```

Player 2 (P2):

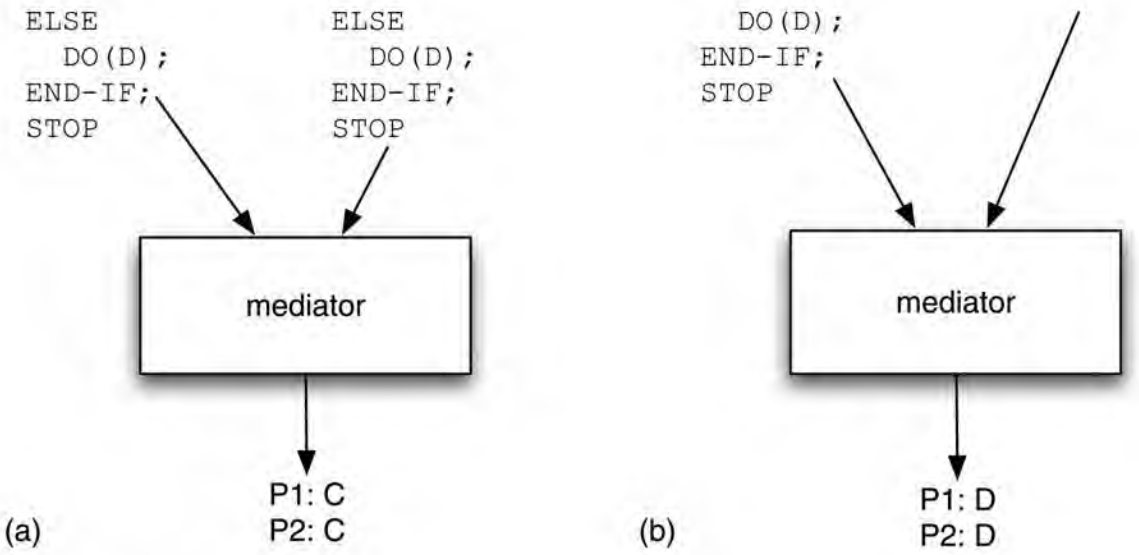
```
IF P1 == P2
THEN
  DO(C);
```

Player 1 (P1):

```
IF P1 == P2 THEN
  DO(C);
ELSE
```

Player 2 (P2):

```
DO(D);
STOP
```




```
IF P1 == P2 THEN
   DO (C);
ELSE
   DO (D);
END-IF;
STOP
```

So, what should player 2 do? The way this program is constructed, if player 2 does *anything other than submit exactly the same program*, then player 1 will execute `DO(D)`, i.e. will defect. I hope you can see that the best response for player 2 is therefore to submit exactly the same program. In this case, both players will end up executing the `DO(C)` action, i.e. will cooperate, yielding a pay-off of 3 to both players. Crucially, *there is no risk of getting the sucker's pay-off by submitting the above program*. Assuming that you submit the above program, then the other player's best response is to submit the same program; and if the other player submits the above program, the best you can do is also submit the above program. In other words, we have a kind of Nash equilibrium, which in this special setting is called a *program equilibrium*. The program equilibrium is a new concept, and there are several points to make:

- If you know a little about theoretical computer science, then you might be wondering whether it is necessary to have programs compared as strings – this seems rather strict. Surely it would be more attractive to be able to say 'if the *behaviour* of the other program is the same as mine, then cooperate, else defect'. But if you really do know something about theoretical computer science, then you should see the difficulty with this idea: it involves reasoning about the behaviour of programs, which is computationally very complex even under very restrictive assumptions, and computationally impossible (undecidable) in general. Using textual comparison to compare programs is a safe and computationally simple compromise – albeit a theoretically inelegant one!
- It is natural to worry about the role of the mediator: the program that takes all the player programs and executes them. Does the scheme rely on the honesty of the mediator? Doesn't the mediator create a bottleneck? The answer to the first question is probably no; if we can all see the programs that are submitted, then there does not seem to be too much scope for dishonesty, since we can all figure out the result for ourselves, and

thereby verify it. With respect to the second question, the answer is yes, possibly.

At the time of writing, program equilibria and mediators are the subject of much ongoing research.

11.6 Other Symmetric 2 × 2 Interactions

From the amount of space we have devoted to discussing it, you might assume that the prisoner's dilemma was the *only* type of multiagent interaction there is. This is, of course, not the case! Recall the ordering of agent i's preferences in the prisoner's dilemma:

$$(D, C) \succ_i (C, C) \succ_i (D, D) \succ_i (C, D).$$

This is just one of the possible orderings of outcomes that agents may have. If we restrict our attention to interactions in which there are two agents, each agent has two possible actions (C or D), and the scenario is *symmetric*, then there are 4! = 24 possible orderings of preferences, which for completeness I have summarized in Table 11.1. (In the game-theory literature, these are referred to as symmetric 2 × 2 games.)

In many of these scenarios, what an agent should do is clear cut. For example, agent i should clearly cooperate in scenarios (1) and (2), as both of the outcomes in which i cooperates are preferred over both of the outcomes in which i defects. Similarly, in scenarios (23) and (24), agent i should clearly defect, as both outcomes in which it defects are preferred over both outcomes in which it cooperates. Scenario (14) is the prisoner's dilemma, which we have already discussed at length. This leaves us with two other interesting cases to examine: the *stag hunt* and the *game of chicken*.

The stag hunt

The stag hunt is another example of a social dilemma. The name stag hunt arises from a scenario put forward by the Swiss philosopher Jean-Jacques Rousseau in his 1775 *Discourse on Inequality*. However, to explain the dilemma, I will use a scenario that will perhaps be more relevant to readers at the beginning of the 21st century [Poundstone, 1992, pp. 218–219].

STAG HUNT

Table 11.1: The possible preferences that agent i can have in symmetric interaction scenarios where there are two agents, each of which has two available actions, C (cooperate) and D (defect); recall that X, Y means the outcome in which agent i plays X and agent j plays Y.

Scenario	Preferences over outcomes	Comment
1.	$(C, C) \succ_i (C, D) \succ_i (D, C) \succ_i (D, D)$	cooperation dominates
2.	$(C, C) \succ_i (C, D) \succ_i (D, D) \succ_i (D, C)$	cooperation dominates
3.	$(C, C) \succ_i (D, C) \succ_i (C, D) \succ_i (D, D)$	
4.	$(C, C) \succ_i (D, C) \succ_i (D, D) \succ_i (C, D)$	stag hunt
5.	$(C, C) \succ_i (D, D) \succ_i (C, D) \succ_i (D, C)$	

6.	$(C, C) \succ_i (D, D) \succ_i (D, C) \succ_i (C, D)$	
7.	$(C, D) \succ_i (C, C) \succ_i (D, C) \succ_i (D, D)$	
8.	$(C, D) \succ_i (C, C) \succ_i (D, D) \succ_i (D, C)$	
9.	$(C, D) \succ_i (D, C) \succ_i (C, C) \succ_i (D, D)$	
10.	$(C, D) \succ_i (D, C) \succ_i (D, D) \succ_i (C, C)$	
11.	$(C, D) \succ_i (D, D) \succ_i (C, C) \succ_i (D, C)$	
12.	$(C, D) \succ_i (D, D) \succ_i (D, C) \succ_i (C, C)$	
13.	$(D, C) \succ_i (C, C) \succ_i (C, D) \succ_i (D, D)$	game of chicken
14.	$(D, C) \succ_i (C, C) \succ_i (D, D) \succ_i (C, D)$	prisoner's dilemma
15.	$(D, C) \succ_i (C, D) \succ_i (C, C) \succ_i (D, D)$	
16.	$(D, C) \succ_i (C, D) \succ_i (D, D) \succ_i (C, C)$	
17.	$(D, C) \succ_i (D, D) \succ_i (C, C) \succ_i (C, D)$	
18.	$(D, C) \succ_i (D, D) \succ_i (C, D) \succ_i (C, C)$	
19.	$(D, D) \succ_i (C, C) \succ_i (C, D) \succ_i (D, C)$	
20.	$(D, D) \succ_i (C, C) \succ_i (D, C) \succ_i (C, D)$	
21.	$(D, D) \succ_i (C, D) \succ_i (C, C) \succ_i (D, C)$	
22.	$(D, D) \succ_i (C, D) \succ_i (D, C) \succ_i (C, C)$	
23.	$(D, D) \succ_i (D, C) \succ_i (C, C) \succ_i (C, D)$	defection dominates
24.	$(D, D) \succ_i (D, C) \succ_i (C, D) \succ_i (C, C)$	defection dominates

You and a friend decide it would be a great joke to show up on the last day of school with some ridiculous haircut. Egged on by your clique, you both *swear* you'll get the haircut.

A night of indecision follows. As you anticipate your parents' and teachers' reactions, you start wondering if your friend is really going to go through with the plan.

Not that you do not want the plan to succeed: the best possible outcome would be for both of you to get the haircut.

The trouble is, it would be awful to be the *only* one to show up with the haircut. That would be the worst possible outcome.

You're not above enjoying your friend's embarrassment. If you *didn't* get the haircut, but the friend did, and looked like a real jerk, that would be almost as good as if you both got the haircut.

This scenario is obviously very close to the prisoner's dilemma: the difference is that in this scenario, mutual cooperation is the most preferred outcome, rather than you defecting while your opponent cooperates. Expressing the game in a pay-off matrix (picking rather arbitrary pay-offs to give the preferences):

	i defects	*i* cooperates
j defects	1 1	2 0
j	0 2	

cooperates	3		3

It should be clear that there are *two* Nash equilibria in this game: mutual defection, or mutual cooperation. If you trust your opponent, and believe that he will cooperate, then you can do no better than cooperate, and vice versa, your opponent can also do no better than cooperate. Conversely, if you believe your opponent will defect, then you can do no better than defect yourself, and vice versa.

Poundstone suggests that 'mutiny' scenarios are examples of the stag hunt: 'We'd all be better off if we got rid of Captain Bligh, but we'll be hung as mutineers if not enough of us go along' [Poundstone, 1992, p. 220].

The game of chicken

The game of chicken (row 13 in Table 11.1) is characterized by agent i having the following preferences:

GAME OF CHICKEN

$$(D, C) \succ_i (C, C) \succ_i (C, D) \succ_i (D, D).$$

As with the stag hunt, this game is also closely related to the prisoner's dilemma. The difference here is that mutual defection is agent i's most feared outcome, rather than i cooperating while j defects. The game of chicken gets its name from a rather silly, macho 'game' that was supposedly popular among juvenile delinquents in 1950s America; the game was immortalized by James Dean in the film *Rebel Without a Cause*. The purpose of the game is to establish who is the braver of two young thugs. The game is played by both players driving their cars at high speed towards a cliff. The idea is that the least brave of the two (the 'chicken') will be the first to drop out of the game by steering away from the cliff. The winner is the one who lasts longest in the car. Of course, if *neither* player steers away, then both cars fly off the cliff, taking their foolish passengers to a fiery death on the rocks that undoubtedly lie below.

So, how should agent i play this game? It depends on how brave (or foolish) i believes j is. If i believes that j is braver than i, then i would do best to steer away from the cliff (i.e. cooperate), since it is unlikely that j will steer away from the cliff. However, if i believes that j is *less* brave than i, then i should stay in the car; because j is less brave, he will steer away first, allowing i to win. The difficulty arises when both agents mistakenly believe that the other is less brave; in this case, both agents will stay in their cars (i.e. defect), and the worst outcome arises.

Expressed as a pay-off matrix, the game of chicken is as follows:

	i defects		i cooperates
j defects	0		0
	3		1
j	1		3
cooperates	2		2

It should be clear that the game of chicken has two Nash equilibria, corresponding to the above-right and below-left cells. Thus, if you believe that your opponent is going to drive straight (i.e. defect), then you can do no better than to steer away from the cliff, and vice versa. Similarly, if you believe that your opponent is going to steer away, then you can do no

versa. Similarly, if you believe that your opponent is going to steer away, then you can do no better than to drive straight.

11.7 Representing Multiagent Scenarios

Suppose a group of agents are to interact with one another in some game-like scenario. How do these agents obtain a common understanding of the rules of the game, and the effects that their actions have? There are two possibilities:

- The rules are hardwired into the participants at design time.
- The rules are specified in some computer-processable format, so that these rules can be understood by software players *at run-time*.

The first option is very limiting, for rather obvious reasons: it requires knowledge of all possible types of encounter at design, and if a player ever encounters a situation that was not considered by the designer of that player, then the player is unable to participate. The second possibility is much more attractive conceptually, although there are formidable obstacles to be overcome in making it a reality. Some first steps towards this vision were taken with the development of the *game description language* (GDL).

GENERAL GAME PLAYING

GDL has its origins in the *general game playing competition*, which was introduced in 2005 as a way of testing *the ability of computer programs to play games in general*, rather than just the ability to play a single game such as chess [Genereseth and Love, 2005; Pell, 1993]. Participants in the general game playing competition are computer programs that are provided with the rules to previously unknown games during the competition itself; they are required to play these games, and the overall competition winner is the one that fares best overall. Participant programs must interpret the rules of the games *themselves* – without human intervention. The GDL was developed in order to define the games to be played. Thus, a participant program must be able to interpret game definitions expressed in GDL, and then play the game defined by the GDL specification.

GDL is a logic-based language, which contains:

1. general facts about the game that remain true throughout the game
2. facts defining the initial state of the game
3. rules defining the legality of moves in different game configurations
4. rules defining what it means to 'win'
5. rules defining when a game is over.

Figure 11.4 shows a version of the 'Tic-Tac-Toe' game, defined in GDL. In this game, two players take turns to mark a 3 × 3 grid, and the player who succeeds in placing three of its marks in a row, column, or diagonal wins. GDL uses a prefix rule notation based on LISP. The first two lines, (`role xplayer`) and (`role oplayer`), define the two players in this game. The following `init` lines define facts true in the initial state of the game (all the cells are blank, and `xplayer` has the control of the game). The following rule defines the effect of performing an action: if cell (*m, n*) is blank (`cell ?m ?n b`), and `xplayer` marks cell (*m, n*), then in the `next` state, it will be true that cell (*m, n*) is marked by `x`. The next rule says that if the current state is in control of `xplayer`, then the next state will be in control of

`oplayer`. There are rules that define what it means to have a `line` of symbols. The first rule in the second column defines when it is `legal` for a player `?w` to perform a mark action. The `goal` rule defines the aim of the game: it says that the `xplayer` will get a reward of 100 if it brings about a line marked by `x`. The final `terminal` rules define when the game is over. Overall, then, a GDL definition consists of a list of rules, defining the initial state, global properties of the game and the transitions that can be made in the game.

11.8 Dependence Relations in Multiagent Systems

Before leaving the issue of interactions, we briefly discuss another approach to understanding how the properties of a multiagent system can be understood. This approach, due to Sichman and colleagues, attempts to understand the *dependencies* between agents [Sichman and Demazeau, 1995; Sichman et al., 1994]. The basic idea is that a dependence relation exists between two agents if one of the agents requires the other in order to achieve one of its goals. There are a number of possible dependency relations.

Independence There is no dependency between the agents.

Unilateral One agent depends on the other, but not vice versa.

Mutual Both agents depend on each other with respect to the same goal.

Reciprocal dependence The first agent depends on the other for some goal, while the second depends on the first for some goal (the two goals are not necessarily the same). Note that mutual dependence implies reciprocal dependence.

Figure 11.4: A fragment of a game in the game description language (GDL).

```
(role xplayer)
(role oplayer)
(init (cell 1 1 b))
...
(init (cell 3 3 b))
(init (control xplayer))
(<= (next (cell ?m ?n x))
    (does xplayer (mark ?m ?n))
    (true (cell ?m ?n b)))
...
(<= (next (control oplayer))
    (true (control xplayer)))
(<= (line ?m ?x)
    (true (cell ?m 1 ?x))
    (true (cell ?m 2 ?x))
    (true (cell ?m 3 ?x)))

(<= (legal ?w (mark ?x ?y))
    (true (cell ?x ?y b))
    (true (control ?w)))
(<= (legal oplayer noop)
    (true (control xplayer)))
...
(<= (goal xplayer 100)
    (line x))
```

```
    (line x))
...
(<= terminal
    (line x))
(<= terminal
    (line o))
```

These relationships may be qualified by whether or not they are *locally believed* or *mutually believed*. There is a locally believed dependence if one agent believes the dependence exists, but does not believe that the other agent believes it exists. A mutually believed dependence exists when the agent believes the dependence exists, and also believes that the other agent is aware of it. Sichman and colleagues implemented a *social reasoning system* called DepNet [Sichman et al., 1994]. Given a description of a multiagent system, DepNet was capable of computing the relationships that existed between agents in the system.

SOCIAL REASONING

Notes and Further Reading

In the first edition of this textbook, I only cited one text on game theory, Ken Binmore's lucid *Fun and Games* [Binmore, 1992]. Given the number of excellent texts available on this subject, citing only one book seems, in retrospect, a little bit mean. I now have a whole shelf of books on game theory in my office, and my favourites among these are as follows. First, the very recent [Shoham and Leyton-Brown, 2008] gives a thorough grounding in computational aspects of game theory: this is a superb text, and if you are likely to carry out research in game-theoretic aspects of multiagent systems, then I wholeheartedly recommend starting with this. With respect to the more conventional game-theory literature, I find [Osborne and Rubinstein, 1994] to be crisp and precise; it really does a clear job of presenting the key definitions, concepts, and results. A very good, less formal text to accompany it is [Osborne, 2004]. Other less technical introductions are [Dixit and Skeath, 2004; Dutta, 1999]. An entertaining and largely non-mathematical introduction to game theory may be found in [Binmore, 2007]. A non-mathematical introduction to game theory, with an emphasis on the applications of game theory in the social sciences, is [Zagare, 1984].

For a high-level description of the proof that finding mixed strategy Nash equilibria is PPAD-complete, see [Papadimitriou, 2007] (which includes an overview of the class PPAD). For overviews of algorithms for actually computing Nash equilibria, see [McKelvey and McLennan, 1996; von Stengel, 2007].

There is a big literature on the prisoner's dilemma, which goes well beyond game theory; the philosophical implications of the prisoner's dilemma are discussed at length in [Binmore, 1992, pp. 310–316] and [Binmore, 1994, 1998]. There are many other interesting aspects of Axelrod's tournaments. The first is that of *noise*. I mentioned above that the iterated prisoner's dilemma is predicated on the assumption that the participating agents can see the move made by their opponent: they can see, in other words, whether their opponent defects or cooperates. But suppose the game allows for a certain probability that on any given round, an agent will *misinterpret* the actions of its opponent, and perceive cooperation to be defection and vice versa. Suppose two agents are playing the iterated prisoner's dilemma against one another, and both are playing TIT-FOR-TAT. Then both agents will start by cooperating, and in the absence of noise, will continue to enjoy the fruits of mutual